

Kitsune: Một quần thể Autoencoder cho phát hiện xâm nhập mạng trực tuyến

Yisroel Mirsky, Tomer Doitshman, Yuval Elovici và Asaf Shabtai

Bản dịch tiếng Việt

Đơn vị: Đại học Ben-Gurion vùng Negev
Email: {yisroel, tomerdoi}@post.bgu.ac.il, {elovici, shabtaia}@bg.ac.il

Ghi chú bản dịch: Các ký hiệu toán học, tên thuật toán, tên bộ dữ liệu và danh mục tài liệu tham khảo được giữ nguyên khi cần thiết để bảo đảm đối chiếu chính xác với bản gốc. Danh mục tài liệu tham khảo ở cuối bài được giữ bằng ngôn ngữ gốc.

Tóm tắt

Mạng nơ-ron đang trở thành một giải pháp ngày càng phổ biến cho các hệ thống phát hiện xâm nhập mạng (Network Intrusion Detection System - NIDS). Khả năng học những mẫu và hành vi phức tạp giúp mạng nơ-ron phù hợp để phân biệt lưu lượng bình thường với các cuộc tấn công mạng. Tuy nhiên, một nhược điểm của mạng nơ-ron là lượng tài nguyên cần thiết để huấn luyện chúng. Nhiều thiết bị cổng mạng và bộ định tuyến - những nơi có thể triển khai NIDS - đơn giản là không có đủ bộ nhớ hoặc năng lực xử lý để huấn luyện, và đôi khi thậm chí không đủ khả năng thực thi các mô hình như vậy. Quan trọng hơn, các giải pháp mạng nơ-ron hiện có thường được huấn luyện theo phương thức có giám sát. Điều đó có nghĩa là chuyên gia phải gán nhãn lưu lượng mạng và định kỳ cập nhật mô hình bằng tay.

Trong bài báo này, chúng tôi trình bày **Kitsune**, một NIDS cảm-là-chạy có khả năng tự học để phát hiện các cuộc tấn công trong mạng cục bộ mà không cần giám sát, đồng thời hoạt động hiệu quả theo phương thức trực tuyến. Thuật toán lõi của Kitsune, **KitNET**, sử dụng một quần thể các mạng nơ-ron được gọi là autoencoder để cùng phân biệt mẫu lưu lượng bình thường và bất thường. KitNET được hỗ trợ bởi một khung trích xuất đặc trưng có khả năng theo dõi hiệu quả mẫu hành vi của từng kênh mạng. Kết quả đánh giá cho thấy Kitsune có thể phát hiện nhiều loại tấn công với hiệu năng tương đương các bộ phát hiện bất thường ngoại tuyến, ngay cả khi chạy trên Raspberry Pi. Điều này chứng minh rằng Kitsune có thể là một NIDS thực tiễn và tiết kiệm chi phí.

Từ khóa: Phát hiện bất thường, phát hiện xâm nhập mạng, thuật toán trực tuyến, autoencoder, học quần thể.

I. GIỚI THIỆU

Số lượng các cuộc tấn công vào mạng máy tính đã gia tăng qua nhiều năm [1]. Một hệ thống bảo mật thường được dùng để bảo vệ mạng là hệ thống phát hiện xâm nhập mạng (NIDS). NIDS là một thiết bị hoặc phần mềm giám sát toàn bộ lưu lượng đi qua một vị trí chiến lược nhằm phát hiện hoạt động độc hại. Khi phát hiện hoạt động như vậy, hệ thống tạo cảnh báo và gửi đến quản trị viên. Theo cách triển khai truyền thống, NIDS được đặt tại một điểm duy nhất, chẳng hạn cổng Internet. Chiến lược triển khai tại một điểm có thể phát hiện lưu lượng độc hại đi vào hoặc đi ra khỏi mạng, nhưng không phát hiện được lưu lượng độc hại di chuyển bên trong chính mạng đó. Để khắc phục vấn đề này, có thể sử dụng chiến lược triển khai phân tán, trong đó nhiều NIDS được kết nối với một tập hợp các bộ định tuyến và cổng mạng chiến lược trong mạng.

Trong thập niên gần đây, nhiều kỹ thuật học máy đã được đề xuất nhằm cải thiện hiệu năng phát hiện [2], [3], [4]. Một hướng phổ biến là sử dụng mạng nơ-ron nhân tạo (Artificial Neural Network - ANN) để kiểm tra lưu lượng mạng. Lợi ích của ANN là khả năng học các khái niệm phi tuyến phức tạp trong dữ liệu đầu vào. Nhờ đó, ANN có lợi thế lớn về hiệu năng phát hiện so với nhiều thuật toán học máy khác [5], [2].

Cách phổ biến để sử dụng ANN làm NIDS là huấn luyện nó phân loại lưu lượng mạng thành bình thường hoặc thuộc một lớp tấn công nào đó [6], [7], [8]. Quy trình điển hình khi dùng bộ phân loại dựa trên ANN trong chiến lược triển khai tại một điểm gồm:

1. Chuyên gia thu thập một bộ dữ liệu chứa cả lưu lượng bình thường và lưu lượng tấn công.
2. Huấn luyện ANN bằng CPU hoặc GPU mạnh để phân biệt lưu lượng bình thường và lưu lượng tấn công.
3. Chuyển một bản sao của mô hình đã huấn luyện đến NIDS của mạng hoặc tổ chức.
4. Để NIDS thực thi mô hình đã huấn luyện trên lưu lượng mạng quan sát được.

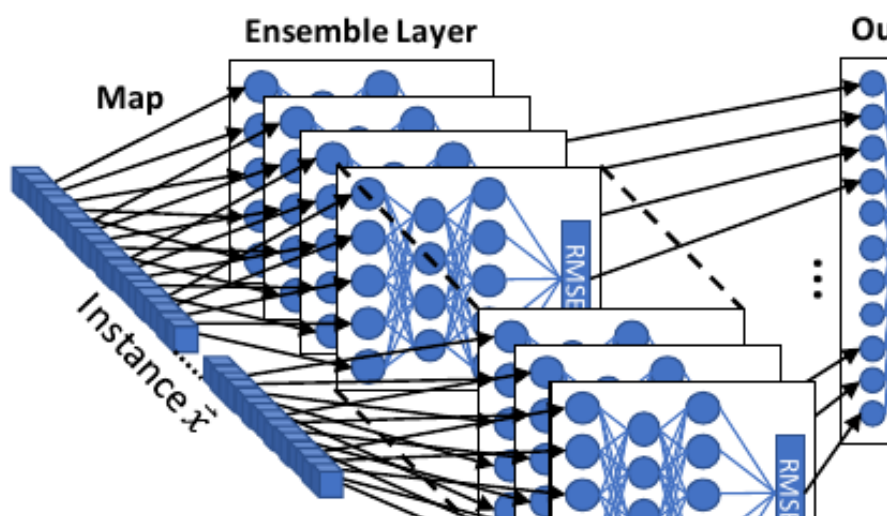
Nhìn chung, chiến lược triển khai phân tán chỉ khả thi khi số lượng NIDS có thể mở rộng theo quy mô mạng với chi phí hợp lý. Một cách để đạt mục tiêu này là nhúng NIDS trực tiếp vào các bộ định tuyến giá rẻ, tức các thiết bị có phần cứng đơn giản. Chúng tôi cho rằng sử dụng các bộ phân loại ANN theo cách này là không thực tế vì các lý do sau.

Xử lý ngoại tuyến. Để huấn luyện một mô hình có giám sát, toàn bộ mẫu đã gán nhãn phải có sẵn tại thiết bị. Điều này không khả thi trên một công mạng đơn giản vì chỉ một giờ lưu lượng có thể chứa hàng triệu gói tin. Một số công trình đề xuất chuyển dữ liệu đến máy chủ từ xa để huấn luyện mô hình [9], [3]. Tuy nhiên, giải pháp đó có thể tạo ra chi phí truyền tải mạng đáng kể và khó mở rộng.

Học có giám sát. Quá trình gán nhãn tốn thời gian và chi phí. Quan trọng hơn, khái niệm “bình thường” phụ thuộc vào lưu lượng cục bộ mà NIDS quan sát được. Ngoài ra, các cuộc tấn công thay đổi theo thời gian và các tấn công mới liên tục được phát hiện [10], vì vậy việc duy trì liên tục một kho lưu lượng tấn công độc hại có thể không thực tế. Cuối cùng, phân loại là một cách tiếp cận theo thể giới đóng: bộ phân loại được huấn luyện để nhận diện các lớp đã xuất hiện trong tập huấn luyện. Không hợp lý khi giả định rằng có thể thu thập mọi lớp lưu lượng độc hại và đưa chúng vào dữ liệu huấn luyện.

Độ phức tạp cao. Độ phức tạp tính toán của ANN tăng rất nhanh theo số lượng nơ-ron [11]. Vì vậy, một ANN được triển khai trên công mạng đơn giản bị giới hạn về kiến trúc và số lượng đặc trưng đầu vào. Vấn đề này đặc biệt nghiêm trọng đối với các công cụ xử lý lưu lượng tốc độ cao.

Hình 1. Minh họa thuật toán phát hiện bất thường KitNET của Kitsune. Các đặc trưng đầu vào được ánh xạ đến **tầng quần thể**; mỗi autoencoder tạo một sai số RMSE; các sai số này được đưa đến **tầng đầu ra** để tạo điểm bất thường cuối cùng.



Từ các thách thức trên, chúng tôi cho rằng một bộ phát hiện xâm nhập dựa trên ANN, được triển khai và huấn luyện phân tán trên các bộ định tuyến, cần đáp ứng các ràng buộc sau:

Xử lý trực tuyến. Sau khi mô hình huấn luyện hoặc thực thi trên một mẫu, mẫu đó được loại bỏ ngay lập tức. Trong thực tế có thể lưu tạm một số lượng nhỏ mẫu tại một thời điểm, tương tự phương pháp phân cụm dòng dữ liệu [12].

Học không giám sát. Các nhân chỉ rõ một gói tin là độc hại hay lành tính không được sử dụng trong quá trình huấn luyện. Có thể dùng thông tin siêu dữ liệu khác, miễn là việc thu thập thông tin đó không làm chậm quá trình xử lý.

Độ phức tạp thấp. Tốc độ xử lý gói tin phải cao hơn tốc độ gói tin đến tối đa dự kiến. Nói cách khác, không được để hình thành hàng đợi gói tin đang chờ mô hình xử lý.

Trong bài báo này, chúng tôi trình bày Kitsune: một NIDS dựa trên ANN mới, hoạt động trực tuyến, không giám sát và hiệu quả. Trong văn hóa dân gian Nhật Bản, Kitsune là sinh vật thần thoại giống cáo, có nhiều đuôi, có khả năng biến đổi hình dạng và trở nên mạnh hơn khi tích lũy kinh nghiệm. Tương tự, Kitsune gồm một quần thể các mạng nơ-ron nhỏ (autoencoder), được huấn luyện để bắt chước - tức tái tạo - các mẫu lưu lượng mạng, và hiệu năng của chúng tăng dần theo thời gian.

Kiến trúc thuật toán phát hiện bất thường của Kitsune (KitNET) được minh họa trong Hình 1. Trước hết, các đặc trưng của một mẫu được ánh xạ đến các nơ-ron nhìn thấy của quần thể. Tiếp theo, mỗi autoencoder cố gắng tái tạo các đặc trưng của mẫu và tính sai số tái tạo bằng căn sai số bình phương trung bình (RMSE). Cuối cùng, các giá trị RMSE được chuyển đến một autoencoder đầu ra, đóng vai trò như cơ chế bỏ phiếu phi tuyến cho toàn quần thể. Trong quá trình huấn luyện Kitsune, tại mỗi thời điểm không có quá một mẫu được lưu trong bộ nhớ. KitNET có một tham số chính: số đầu vào tối đa của mỗi autoencoder trong quần thể. Tham số này giúp tăng tốc thuật toán với sự đánh đổi vừa phải về hiệu năng phát hiện.

Chúng tôi sử dụng autoencoder vì: (1) chúng có thể được huấn luyện theo phương thức không giám sát; và (2) chúng có thể phát hiện bất thường khi tái tạo kém. Chúng tôi đề xuất dùng một quần thể nhiều autoencoder nhỏ vì chúng hiệu quả hơn và có thể ít nhiều hơn một autoencoder duy nhất trên cùng không gian đặc trưng. Thực nghiệm cho thấy Kitsune có thể tăng tốc độ xử lý gói tin lên năm lần, đồng thời đạt hiệu năng phát hiện cạnh tranh với các bộ phát hiện bất thường ngoại tuyến theo lô.

Tóm lại, các đóng góp của bài báo gồm:

- Đề xuất một NIDS mới dựa trên autoencoder dành cho thiết bị mạng đơn giản, có tên Kitsune, với đặc điểm nhẹ và cắm-là-chạy. Theo hiểu biết của chúng tôi, đây là công trình đầu tiên đề xuất sử dụng autoencoder, có hoặc không có quần thể, để phát hiện bất thường trực tuyến trong mạng máy tính. Chúng tôi cũng trình bày thuật toán lõi KitNET như một thuật toán phát hiện bất thường trực tuyến, không giám sát và có tính tổng quát, đồng thời cung cấp mã nguồn để tải về.¹
- Xây dựng một khung trích xuất đặc trưng để duy trì và trích xuất động các đặc trưng ngữ cảnh ngầm từ lưu lượng mạng. Khung này sử dụng ít bộ nhớ vì các thống kê được cập nhật gia tăng trên các cửa sổ có suy giảm.
- Đề xuất kỹ thuật trực tuyến để tự động xây dựng quần thể autoencoder - tức ánh xạ đặc trưng vào đầu vào ANN - theo phương thức không giám sát. Phương pháp sử dụng phân cụm phân cấp gia tăng trên không gian đặc trưng (chuyển vị của bộ dữ liệu không bị giới hạn) và giới hạn kích thước cụm.
- Cung cấp kết quả thực nghiệm trên một mạng giám sát video bằng camera IP đang hoạt động, một mạng IoT và nhiều loại tấn công. Chúng tôi cũng chứng minh hiệu quả của thuật toán và khả năng chạy trên một bộ định tuyến đơn giản thông qua phép đo chuẩn trên Raspberry Pi.

Phần còn lại của bài báo được tổ chức như sau. Phần II thảo luận các công trình liên quan về phát hiện bất thường trực tuyến. Phần III cung cấp kiến thức nền về autoencoder và cách chúng hoạt động. Phần IV trình bày khung Kitsune và toàn bộ pipeline học máy. Phần V trình bày kết quả thực nghiệm về hiệu năng phát hiện và hiệu năng thời gian chạy. Phần VI thảo luận các tấn công đối kháng và biện pháp ứng phó. Cuối cùng, Phần VII đưa ra kết luận.

¹ Mã nguồn KitNET: <https://github.com/ymirsky/KitNET-py>.

II. CÔNG TRÌNH LIÊN QUAN

Việc sử dụng học máy, đặc biệt là phát hiện bất thường, để triển khai NIDS đã được nghiên cứu rộng rãi [13]-[17]. Tuy nhiên, các giải pháp đó thường không đặt giả định rõ ràng về tài nguyên của thiết bị dùng để huấn luyện hoặc thực thi mô hình. Vì vậy, chúng hoặc quá tốn kém để huấn luyện và chạy trên các cổng mạng đơn giản, hoặc cần một bộ dữ liệu đã gán nhãn để huấn luyện.

Một số công trình trước đây đề xuất cơ chế phát hiện bất thường trực tuyến bằng các thuật toán nhẹ khác nhau. Ví dụ, PAYL IDS mô hình hóa các histogram đơn giản của nội dung gói tin [18], hoặc phương pháp sử dụng thuật toán kNN [19]. Những phương pháp này hoặc quá đơn giản nên cho kết quả kém, hoặc yêu cầu tích lũy dữ liệu để huấn luyện hoặc phát hiện.

ANN là một thuật toán phổ biến trong phát hiện xâm nhập mạng vì có khả năng học các khái niệm phức tạp, bao gồm những khái niệm thuộc miền truyền thông mạng [17]. Trong [20], các tác giả đánh giá ANN cùng nhiều thuật toán phân loại khác cho bài toán phát hiện xâm nhập mạng và đề xuất giải pháp dựa trên một quần thể bộ phân loại sử dụng đặc trưng theo kết nối. Trong [8], các tác giả sửa đổi thuật toán lan truyền ngược để tăng tốc quá trình huấn luyện ANN. Trong [7], nhiều bộ phân loại dựa trên ANN được sử dụng, mỗi bộ được huấn luyện để phát hiện một loại tấn công cụ thể. Trong [9], các tác giả đề xuất phương pháp phân cấp: mỗi gói tin trước tiên đi qua mô hình phát hiện bất thường; nếu xuất hiện bất thường, gói tin tiếp tục được đánh giá bởi một tập các bộ phân loại ANN, trong đó mỗi bộ phân loại được huấn luyện cho một loại tấn công cụ thể.

Tất cả các công trình nêu trên sử dụng ANN hoặc là có giám sát, hoặc không phù hợp với cổng mạng đơn giản. Một số công trình còn giả định rằng dữ liệu huấn luyện có thể được lưu trữ và tích lũy, điều không phù hợp với các cổng mạng tài nguyên hạn chế. Giải pháp của chúng tôi cho phép triển khai cắm-là-chạy và có thể hoạt động nhanh hơn đáng kể so với các mô hình trên.

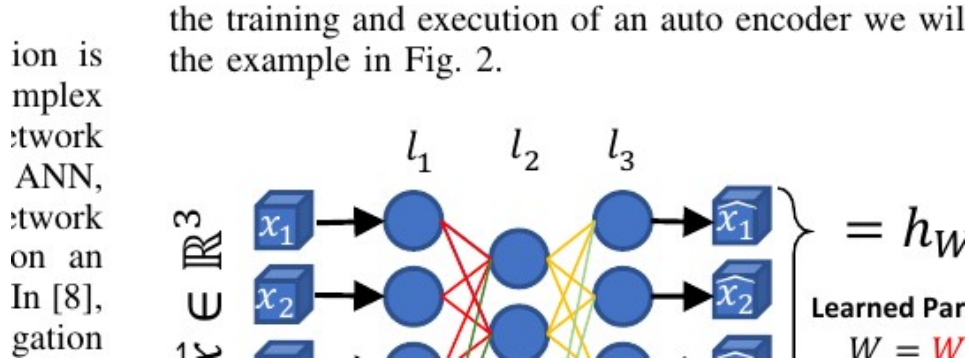
Về việc sử dụng autoencoder: trong [21], các tác giả dùng một quần thể mạng nơ-ron sâu để theo dõi đối tượng trong bối cảnh trực tuyến. Phương pháp sử dụng stacked denoising autoencoder (SDAE). Mỗi tầng của SDAE đóng vai trò như một không gian đặc trưng khác nhau cho dữ liệu ảnh thô. Phương án này chuyển từng tầng của SDAE thành một mạng nơ-ron sâu dùng làm bộ phân loại nhị phân phân biệt. Mặc dù autoencoder được áp dụng trong bối cảnh trực tuyến, công trình không thực hiện phát hiện bất thường và cũng không giải quyết thách thức xử lý thời gian thực - vốn đặc biệt khó với mạng nơ-ron sâu. Hơn nữa, huấn luyện mạng sâu rất phức tạp và không thể thực hiện thực tế trên một thiết bị mạng đơn giản. Trong [22] và [23], autoencoder được dùng để trích xuất đặc trưng từ bộ dữ liệu nhằm cải thiện phát hiện đe dọa mạng. Tuy nhiên, bản thân autoencoder không được dùng để phát hiện bất thường; cuối cùng các tác giả vẫn sử dụng bộ phân loại để phát hiện đe dọa. Vì vậy, các giải pháp đó cần chuyên gia gán nhãn mẫu, trong khi giải pháp của chúng tôi không giám sát và cắm-là-chạy.

Trong [24], các tác giả đề xuất sử dụng autoencoder theo cách tổng quát để phát hiện bất thường. Trong [19], autoencoder được dùng để phát hiện bất thường trong lưới điện. Các công trình này khác với chúng tôi vì: (1) không hoạt động trực tuyến; (2) kiến trúc không nhẹ và không có khả năng mở rộng như một quần thể; và (3) chưa được áp dụng cho phát hiện xâm nhập mạng. Một phần đóng góp của bài báo này là khung trích xuất đặc trưng phù hợp, cho phép sử dụng autoencoder trong bối cảnh mạng trực tuyến.

III. KIẾN THỨC NỀN: AUTOENCODER

Autoencoder là thành phần nền tảng của Kitsune. Phần này giới thiệu ngắn gọn autoencoder là gì và hoạt động như thế nào. Việc mô tả huấn luyện và thực thi autoencoder dựa trên ví dụ trong Hình 2.

Hình 2. Ví dụ một autoencoder có một tầng nén, dùng để tái tạo mẫu gồm ba đặc trưng. Đầu vào $\vec{x} \in \mathbb{R}^3$ đi qua tầng ẩn nén rồi được tái tạo thành $\hat{x}_1, \hat{x}_2, \hat{x}_3$. Các tham số học được gồm $W = (W^{(1)}, W^{(2)})$ và $b = (b^{(1)}, b^{(2)})$; trong cấu trúc đối xứng có thể đặt $W^{(2)} = (W^{(1)})^T$.



A. Mạng nơ-ron nhân tạo

ANN gồm nhiều tầng nơ-ron, trong đó các tầng được kết nối tuần tự bằng các liên kết. Mỗi liên kết có trọng số; toàn bộ trọng số xác định các khái niệm mà mô hình đã học. Cụ thể, ký hiệu $l^{(i)}$ là tầng thứ i của ANN và $\|l^{(i)}\|$ là số nơ-ron trong tầng đó. Tổng số tầng của ANN được ký hiệu là L . Các trọng số nối $l^{(i)}$ với $l^{(i+1)}$ được biểu diễn bởi ma trận $W^{(i)}$ có kích thước $\|l^{(i)}\| \times \|l^{(i+1)}\|$, cùng vector độ lệch $\vec{b}^{(i)}$ có $\|l^{(i+1)}\|$ phần tử. Tập hợp toàn bộ tham số được ký hiệu $\theta \equiv (W, b)$, trong đó W và b lần lượt là các trọng số và độ lệch của các tầng. Hình 2 minh họa cách các trọng số tạo nên các liên kết giữa tầng trong ANN.

ANN có hai loại tầng: tầng nhìn thấy và tầng ẩn. Tầng nhìn thấy nhận mẫu đầu vào $\vec{x}$ cùng một biến độ lệch bổ sung có giá trị hằng bằng 1. $\vec{x}$ là vector đặc trưng số mô tả mẫu và thường được chuẩn hóa xấp xỉ trong khoảng $[-1, +1]$, chẳng hạn bằng chuẩn hóa 0-1 hoặc z-score [25]. Khác biệt là tầng nhìn thấy được coi là đã tính sẵn và sẵn sàng truyền đến tầng thứ hai, trong khi các tầng còn lại là tầng ẩn.

B. Thực thi ANN

Để thực thi ANN, tầng $l^{(2)}$ được kích hoạt bằng đầu ra của $l^{(1)}$ - tức $\vec{x}$ - sau khi nhân với $W^{(1)}$. Sau đó đầu ra của $l^{(2)}$ được nhân với $W^{(2)}$ để kích hoạt $l^{(3)}$, và quá trình tiếp tục cho đến khi kích hoạt tầng cuối. Quá trình này gọi là **lan truyền tiến**.

Ký hiệu $\vec{a}^{(i)}$ là vector đầu ra của các nơ-ron trong tầng $l^{(i)}$. Để thu được $\vec{a}^{(i+1)}$, truyền $\vec{a}^{(i)}$ qua tầng $l^{(i+1)}$ bằng:

$$\vec{a}^{(i+1)} = f(W^{(i)} \cdot \vec{a}^{(i)} + \vec{b}^{(i)}).$$

Trong đó f là hàm kích hoạt của nơ-ron. Một hàm kích hoạt phổ biến, đồng thời được sử dụng trong Kitsune, là sigmoid:

$$f(\vec{x}) = \frac{1}{1 + e^{-\vec{x}}}.$$

Đầu ra của tầng cuối được ký hiệu là $\vec{y}' = \vec{a}^{(L)}$. Hàm h biểu diễn toàn bộ quá trình lan truyền tiến theo từng tầng từ đầu vào $\vec{x}$ đến đầu ra cuối $\vec{y}'$:

$$h_{\theta}(\vec{x}) = \vec{y}'.$$

C. Huấn luyện ANN

Đầu ra của ANN phụ thuộc vào tập huấn luyện và thuật toán huấn luyện. Một bộ dữ liệu huấn luyện gồm các mẫu $\vec{x} \in X$ và đầu ra kỳ vọng tương ứng $\vec{y} \in Y$, chẳng hạn như trong bài toán phân loại. Trong quá trình huấn luyện, trọng số của ANN được điều chỉnh để $h_{\theta}(\vec{x}) = \vec{y}$. Thuật toán phổ biến để huấn luyện ANN - tức tìm W và b tối ưu khi biết X, Y - là thuật toán lan truyền ngược [6].

Trong lan truyền ngược, sai số giữa đầu ra dự đoán y' và đầu ra kỳ vọng y được truyền ngược từ đầu ra đến từng nơ-ron. Trong quá trình này, sai số giữa mức kích hoạt thực tế và kỳ vọng của mỗi nơ-ron được lưu lại. Bước lan truyền ngược được ký hiệu là hàm $b_{\theta}(Y, Y')$. Với một lần thực thi h_{θ} , gọi A là toàn bộ mức kích hoạt của nơ-ron và Δ là toàn bộ sai số kích hoạt. Khi biết A, Δ hiện tại và tốc độ học $\ell \in (0, 1]$, có thể tối ưu gia tăng W và b bằng thuật toán hạ gradient (Gradient Descent - GD) [26].

Thuật toán 1. Lan truyền ngược để huấn luyện ANN theo lô

```

procedure trainGD( $\theta, X, Y, \text{max\_iter}$ )
1  $\theta \leftarrow U(-1/\|l(1)\|, 1/\|l(1)\|)$  // khởi tạo ngẫu nhiên
2  $\text{cur\_iter} \leftarrow 0$ 
3 while  $\text{cur\_iter} \leq \text{max\_iter}$  do
4    $A, Y' \leftarrow h_{\theta}(X)$  // lan truyền tiến
5    $\text{deltas} \leftarrow b_{\theta}(Y, Y')$  // lan truyền ngược
6    $\theta \leftarrow \text{GD}_{\ell}(A, \text{deltas})$  // cập nhật trọng số
7    $\text{cur\_iter}++$ 
8 end
9 return  $\theta$ 

```

Quá trình trên được gọi là **huấn luyện theo lô**, vì ở mỗi vòng lặp GD cập nhật trọng số dựa trên sai số tổng hợp của tất cả các mẫu trong X . Một phương án khác là huấn luyện ngẫu nhiên, trong đó sử dụng Stochastic Gradient Descent (SGD) thay cho GD. Với SGD, trọng số được cập nhật theo sai số của từng mẫu riêng lẻ.

Thuật toán 2. Lan truyền ngược để huấn luyện ngẫu nhiên ANN

```

procedure trainSGD( $\theta, X, Y, \text{max\_iter}$ )
1  $\theta \leftarrow U(-1/\|l(1)\|, 1/\|l(1)\|)$  // khởi tạo ngẫu nhiên
2  $\text{cur\_iter} \leftarrow 0$ 
3 while  $\text{cur\_iter} \leq \text{max\_iter}$  do
4   for  $x_i$  in  $X$  do
5      $A, y' \leftarrow h_{\theta}(x_i)$  // lan truyền tiến
6      $\text{deltas} \leftarrow b_{\theta}(y, y')$  // lan truyền ngược
7      $\theta \leftarrow \text{GD}_{\ell}(A, \text{deltas})$  // cập nhật trọng số
8   end
9    $\text{cur\_iter}++$ 
10 end
11 return  $\theta$ 

```

Khác biệt giữa GD và SGD là GD hội tụ đến nghiệm tối ưu tốt hơn, nhưng SGD hội tụ nhanh hơn trong giai đoạn đầu [27]. Giải thích chi tiết hơn về lan truyền ngược có thể xem trong [26].

Trong Kitsune, chúng tôi sử dụng SGD với $\text{max_iter} = 1$. Nói cách khác, trong pha huấn luyện, khi một mẫu mới đến, hệ thống chỉ thực hiện một vòng của vòng lặp bên trong Thuật toán 2, loại bỏ mẫu rồi chờ mẫu tiếp theo. Nhờ đó, hệ thống chỉ học đúng một lần từ mỗi mẫu quan sát được và vẫn là một thuật toán trực tuyến.

D. Autoencoder

Autoencoder là một ANN được huấn luyện để tái tạo chính đầu vào của nó, tức $X=Y$. Cụ thể, trong quá trình huấn luyện, autoencoder cố học hàm:

$$h_{\theta}(\vec{x}) \approx \vec{x}.$$

Có thể thấy autoencoder về bản chất cố học hàm đồng nhất của phân phối dữ liệu gốc. Vì vậy, người ta đặt các ràng buộc lên mạng để buộc nó học những khái niệm và quan hệ có ý nghĩa hơn giữa các đặc trưng trong $\vec{x}$. Ràng buộc phổ biến nhất là giới hạn số nơ-ron ở các tầng bên trong. “Nút thắt” hẹp buộc mạng học cách mã hóa và giải mã gọn các mẫu đầu vào.

Trong ví dụ Hình 2, autoencoder nhận mẫu $\vec{x} \in R^3$ ở tầng $l^{(1)}$, mã hóa - nén - $\vec{x}$ tại tầng $l^{(2)}$, rồi giải mã biểu diễn đã nén tại tầng $l^{(3)}$. Nếu autoencoder đối xứng về kích thước tầng, có thể dùng cùng các trọng số đối xứng cho mã hóa và giải mã [28]. Mẹo này làm giảm số phép tính trong quá trình huấn luyện. Chẳng hạn trong Hình 2, $W^{(2)} = [W^{(1)}]^T$.

E. Phát hiện bất thường bằng autoencoder

Autoencoder đã được dùng cho nhiều tác vụ học máy, chẳng hạn sinh nội dung mới [29] hoặc loại nhiễu khỏi ảnh [30]. Trong bài báo này, chúng tôi quan tâm đến việc dùng autoencoder để phát hiện bất thường.

Nhìn chung, autoencoder được huấn luyện trên X sẽ có khả năng tái tạo các mẫu chưa từng thấy nhưng thuộc cùng phân phối dữ liệu với X . Nếu một mẫu không thuộc những khái niệm học được từ X , sai số tái tạo được kỳ vọng là lớn. Sai số tái tạo của mẫu $\vec{x}$ có thể được tính bằng RMSE giữa $\vec{x}$ và đầu ra tái tạo $\vec{y}'$:

$$RMSE(\vec{x}, \vec{y}') = \sqrt{\frac{\sum_{i=1}^n (x_i - y_i')^2}{n}},$$

trong đó n là số chiều của các vector đầu vào.

Gọi ϕ là ngưỡng bất thường, ban đầu bằng -1 , và $\beta \in \mathbb{R}$ là tham số độ nhạy. Có thể áp dụng autoencoder cho phát hiện bất thường theo các bước:

1. **Pha huấn luyện:** Huấn luyện autoencoder trên dữ liệu sạch (bình thường). Với mỗi mẫu x_i trong tập huấn luyện X :
 - Thực thi: $s = RMSE(\vec{x}, h_{\theta}(\vec{x}))$.
 - Cập nhật: nếu $s \geq \phi$ thì $\phi \leftarrow s$.
 - Huấn luyện: cập nhật θ bằng cách học từ x_i .
2. **Pha thực thi:** Khi một mẫu chưa từng thấy $\vec{x}$ xuất hiện:
 - Thực thi: $s = RMSE(\vec{x}, h_{\theta}(\vec{x}))$.
 - Kết luận: nếu $s \geq \phi \beta$ thì phát cảnh báo.

Quá trình Kitsune phát hiện bất thường trên một quần thể autoencoder được trình bày chi tiết tại Phần IV-D.

F. Độ phức tạp

Để kích hoạt tầng $l^{(i+1)}$, cần thực hiện phép nhân ma trận $W^{(i)} \cdot \vec{a}^{(i)}$ như trong (1). Vì vậy, độ phức tạp để kích hoạt tầng $l^{(i+1)}$ là $O(\|\vec{l}^{(i)}\| \cdot \|\vec{l}^{(i+1)}\|)$.² Tổng độ phức tạp khi thực thi ANN phụ thuộc vào số tầng và số nơ-ron trong từng tầng. Độ phức tạp huấn luyện ANN trên một mẫu bằng SGD xấp xỉ gấp đôi độ phức tạp thực thi do cần thêm bước lan truyền ngược.

Autoencoder có thể là mạng sâu với nhiều tầng ẩn. Nói chung, mạng càng sâu và rộng càng có thể học những khái niệm phức tạp, nhưng cũng tốn kém hơn về tính toán khi huấn luyện và thực thi. Vì vậy, trong KitNET, mỗi autoencoder được giới hạn ở ba tầng và tối đa bảy nơ-ron nhìn thấy.

² Các hệ thống hiện đại thường tăng tốc phép nhân ma trận bằng GPU. Trong bài báo này, chúng tôi giả định không có GPU, như trường hợp của bộ định tuyến mạng đơn giản.

IV. NIDS KITSUNE

Phần này trình bày NIDS Kitsune, gồm khung tiền xử lý gói tin, bộ trích xuất đặc trưng và thuật toán phát hiện bất thường lỗi. Chúng tôi cũng phân tích độ phức tạp của thuật toán phát hiện bất thường và đưa ra giới hạn cho hiệu năng thời gian chạy.

A. Tổng quan

Kitsune là một NIDS cắm-là-chạy dựa trên mạng nơ-ron, được thiết kế để phát hiện hiệu quả các mẫu bất thường trong lưu lượng mạng. Hệ thống hoạt động bằng cách: (1) giám sát các mẫu thống kê của lưu lượng mạng gần đây; và (2) phát hiện mẫu bất thường bằng một quần thể autoencoder. Mỗi autoencoder trong quần thể chịu trách nhiệm phát hiện bất thường liên quan đến một khía cạnh cụ thể của hành vi mạng. Do Kitsune được thiết kế để chạy theo thời gian thực trên các bộ định tuyến đơn giản, hệ thống có mức sử dụng bộ nhớ nhỏ và độ phức tạp tính toán thấp.

Khung Kitsune gồm các thành phần sau:

- **Bộ bắt gói tin (Packet Capturer):** Thư viện bên ngoài chịu trách nhiệm thu nhận gói tin thô. Ví dụ: NFQueue [31], afpacket [32] và tshark [33] - API của Wireshark.
- **Bộ phân tích gói tin (Packet Parser):** Thư viện bên ngoài chịu trách nhiệm phân tích gói tin thô để lấy siêu dữ liệu mà Bộ trích xuất đặc trưng cần. Ví dụ: PcapPlusPlus³ và tshark.
- **Bộ trích xuất đặc trưng (Feature Extractor - FE):** Thành phần trích xuất n đặc trưng từ các gói tin đến để tạo mẫu $\vec{x} \in R^n$. Các đặc trưng của $\vec{x}$ mô tả gói tin và kênh mạng mà gói tin xuất phát.
- **Bộ ánh xạ đặc trưng (Feature Mapper - FM):** Thành phần tạo một tập các mẫu con nhỏ hơn, ký hiệu v , từ $\vec{x}$ rồi chuyển v đến Bộ phát hiện bất thường (AD). FM đồng thời học phép ánh xạ từ $\vec{x}$ sang v .
- **Bộ phát hiện bất thường (Anomaly Detector - AD):** Thành phần phát hiện gói tin bất thường dựa trên biểu diễn v của gói tin.

Do Bộ bắt gói tin và Bộ phân tích gói tin không phải đóng góp chính của bài báo, nội dung sau tập trung vào FE, FM và AD. FM và AD không phụ thuộc vào tác vụ cụ thể mà chỉ phụ thuộc vào các đặc trưng đầu vào, vì vậy có thể tái sử dụng chúng như một thuật toán phát hiện bất thường trực tuyến tổng quát. Thuật toán tổng quát trong AD được gọi là **KitNET**.

KitNET có một tham số đầu vào chính là m : số đầu vào tối đa cho mỗi autoencoder trong quần thể. Tham số này ảnh hưởng đến độ phức tạp của quần thể KitNET. Vì m tạo ra sự đánh đổi giữa hiệu năng phát hiện và hiệu năng thời gian chạy, người dùng Kitsune phải quyết định yếu tố nào quan trọng hơn - tỷ lệ phát hiện hay tốc độ xử lý gói tin. Sự đánh đổi này được thảo luận thêm trong Phần V.

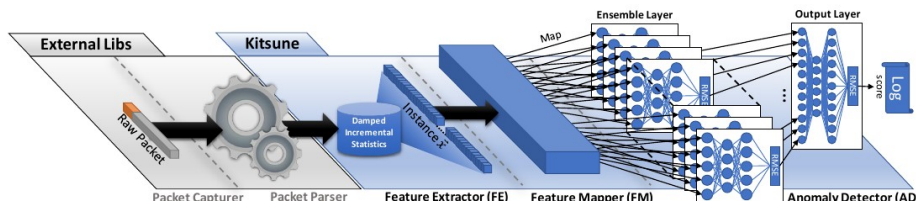
FM và AD có hai chế độ hoạt động: **chế độ huấn luyện** (train-mode) và **chế độ thực thi** (exec-mode). Với cả hai thành phần, chế độ huấn luyện chuyển sang chế độ thực thi sau một khoảng thời gian do người dùng định nghĩa. Trong chế độ huấn luyện, thành phần cập nhật các biến nội bộ từ đầu vào nhưng không tạo đầu ra. Ngược lại, trong chế độ thực thi, thành phần không cập nhật các biến nhưng tạo ra đầu ra.

Để hiểu rõ cách Kitsune hoạt động, quy trình khi một gói tin được thu nhận - minh họa trong Hình 3 - diễn ra như sau:

1. Bộ bắt gói tin thu nhận một gói mới và chuyển dữ liệu nhị phân thô đến Bộ phân tích gói tin.
2. Bộ phân tích gói tin nhận dữ liệu nhị phân thô, phân tích gói và gửi siêu dữ liệu của gói đến FE, chẳng hạn thời điểm đến, kích thước và địa chỉ mạng.

3. FE nhận thông tin này và dùng nó để truy xuất hơn 100 thống kê mô tả ngầm trạng thái hiện tại của kênh mà gói tin xuất phát. Các thống kê tạo thành mẫu $\vec{x} \in R^n$ và được chuyển đến FM.
4. FM nhận $\vec{x}$:
 - **Chế độ huấn luyện:** dùng $\vec{x}$ để học bản đồ đặc trưng. Bản đồ nhóm các đặc trưng của $\vec{x}$ thành nhiều tập, mỗi tập có kích thước tối đa m . Không có dữ liệu nào được chuyển đến AD trước khi bản đồ hoàn tất. Khi kết thúc chế độ huấn luyện, bản đồ được chuyển đến AD; AD dùng bản đồ đó để xây dựng kiến trúc quần thể, trong đó mỗi tập đặc trưng trở thành đầu vào của một autoencoder.
 - **Chế độ thực thi:** dùng phép ánh xạ đã học để tạo tập các mẫu con nhỏ v từ $\vec{x}$, rồi gửi từng mẫu con đến autoencoder tương ứng trong tầng quần thể của AD.
5. AD nhận v :
 - **Chế độ huấn luyện:** dùng v để huấn luyện tầng quần thể. Các RMSE sinh ra trong lan truyền tiến được dùng để huấn luyện tầng đầu ra. RMSE lớn nhất của tầng đầu ra được đặt làm ϕ và lưu lại để dùng sau.
 - **Chế độ thực thi:** thực thi v qua toàn bộ các tầng. Nếu RMSE của tầng đầu ra vượt $\phi \beta$, hệ thống ghi một cảnh báo kèm thông tin chi tiết của gói tin.
6. Gói tin gốc, $\vec{x}$ và v đều được loại bỏ.

Hình 3. Kiến trúc Kitsune. Các thư viện ngoài thực hiện bắt và phân tích gói tin. Bên trong Kitsune, FE duy trì các thống kê gia tăng có suy giảm; FM ánh xạ vector đặc trưng thành nhiều mẫu con; AD gồm tầng quần thể và tầng đầu ra, tạo điểm bất thường rồi ghi log.



³ Dự án PcapPlusPlus: <https://github.com/seladb/PcapPlusPlus>.

B. Bộ trích xuất đặc trưng (FE)

Trích xuất đặc trưng là quá trình thu được hoặc thiết kế một vector giá trị mô tả một quan sát trong thế giới thực. Trong phát hiện bất thường mạng, cần trích xuất các đặc trưng phản ánh ngữ cảnh và mục đích của từng gói tin đi qua mạng. Ví dụ, một gói TCP SYN đơn lẻ có thể là yêu cầu lành tính nhằm thiết lập kết nối với máy chủ, hoặc có thể là một trong hàng triệu gói tương tự được gửi để gây tấn công từ chối dịch vụ (DoS). Một ví dụ khác là luồng video từ camera giám sát IP. Mặc dù nội dung gói tin hợp lệ, độ dao động thời gian truyền (jitter) có thể đột ngột tăng mạnh và duy trì liên tục; hiện tượng này có thể cho thấy lưu lượng đang bị nghe lén trong một cuộc tấn công người-ở-giữa.

Đây là những ví dụ trong đó đặc trưng thống kê theo thời gian có thể giúp phát hiện bất thường. Việc trích xuất loại đặc trưng này từ lưu lượng mạng gặp ba thách thức: (1) gói tin của nhiều kênh hoặc cuộc hội thoại khác nhau bị xen kẽ; (2) tại một thời điểm có thể tồn tại rất nhiều kênh; và (3) tốc độ gói tin đến có thể rất cao. Cách đơn giản là duy trì một cửa sổ gói tin cho từng kênh và liên tục tính thống kê trên các cửa sổ đó. Tuy nhiên, cách này nhanh chóng trở nên không thực tế về bộ nhớ và khó mở rộng.

Vì vậy, chúng tôi thiết kế một khung trích xuất tốc độ cao các thống kê theo thời gian trên số lượng dòng dữ liệu - kênh mạng - thay đổi động. Khung có mức sử dụng bộ nhớ nhỏ vì dùng thống kê gia tăng được duy trì trên cửa sổ có suy giảm. Cửa sổ có suy giảm làm cho đặc trưng mang tính thời gian, tức phản ánh hành vi gần đây của kênh; đồng thời một thống kê gia tăng có thể bị xóa khi trọng số suy giảm của nó về 0, giúp tiết kiệm thêm bộ

nhớ. Khung có độ phức tạp $O(1)$ vì tập các thống kê gia tăng được lưu trong bảng băm. Khung cũng duy trì các thống kê hai chiều hữu ích để phản ánh quan hệ giữa lưu lượng nhận (r_x) và truyền (t_x) của một kết nối.

Sau đây là cách thống kê gia tăng có suy giảm hoạt động, rồi đến các đặc trưng thống kê mà FE trích xuất để tạo mẫu $\tilde{x}$.

1. Thống kê gia tăng có suy giảm

Gọi $S = \{x_1, x_2, \dots\}$ là một dòng dữ liệu không giới hạn, với $x_i \in R$. Chẳng hạn, S có thể là chuỗi kích thước gói tin quan sát được. Trung bình, phương sai và độ lệch chuẩn của S có thể được cập nhật gia tăng bằng cách duy trì bộ:

$$IS = (N, LS, SS),$$

trong đó N , LS và SS lần lượt là số mẫu, tổng tuyến tính và tổng bình phương của các mẫu đã quan sát. Khi chèn x_i :

$$IS \leftarrow (N+1, LS+x_i, SS+x_i^2).$$

Tại mọi thời điểm:

$$\mu_s = \frac{LS}{N}, \sigma_s^2 = \left| \frac{SS}{N} - \left(\frac{LS}{N} \right)^2 \right|, \sigma_s = \sqrt{\sigma_s^2}.$$

Để trích xuất hành vi hiện tại của dòng dữ liệu, cần “quên” các mẫu cũ. Cách đơn giản là duy trì cửa sổ trượt các giá trị, nhưng cách này có độ phức tạp bộ nhớ và thời gian $O(n)$, trong khi thống kê gia tăng chỉ cần $O(1)$. Hơn nữa, cửa sổ trượt không xét khoảng thời gian thực mà cửa sổ bao phủ: 100 mẫu gần nhất có thể đến trong một giờ hoặc chỉ vài giây.

Giải pháp là sử dụng thống kê gia tăng có suy giảm. Trong mô hình cửa sổ suy giảm, trọng số của các giá trị cũ giảm theo hàm mũ theo thời gian. Gọi d là hàm suy giảm:

$$d_\lambda(t) = 2^{-\lambda t},$$

trong đó $\lambda > 0$ là hệ số suy giảm và t là thời gian từ lần quan sát gần nhất của dòng S_i . Bộ thống kê gia tăng có suy giảm được định nghĩa:

$$IS_{i,\lambda} = (w, LS, SS, SR_{ij}, T_{last}),$$

trong đó w là trọng số hiện tại, T_{last} là dấu thời gian của lần cập nhật gần nhất, còn SR_{ij} là tổng tích phần dư giữa hai dòng i và j , dùng để tính thống kê 2D. Để cập nhật $IS_{i,\lambda}$ bằng giá trị x_{cur} tại thời điểm t_{cur} , thực hiện Thuật toán 3.

Thuật toán 3. Chèn giá trị mới vào thống kê gia tăng có suy giảm

```

procedure update(ISi, λ, xcur, tcur, rj)
1  γ ← dλ(tcur - tlast) // tính hệ số suy giảm
2  ISi,λ ← (γw, γLS, γSS, γSR, Tcur) // áp dụng suy giảm
3  ISi,λ ← (w+1, LS+xcur, SS+xcur²,
    SRij + rj*rj, Tcur) // chèn giá trị
4  return ISi,λ

```

Bảng I. Các thống kê gia tăng có thể tính từ S_i và S_j

Loại	Thống kê	Ký hiệu	Cách tính
1D	Trọng số	w	w

Loại	Thống kê	Ký hiệu	Cách tính
1D	Trung bình	μ_{S_i}	LS/w
1D	Độ lệch chuẩn	σ_{S_i}	$\sqrt{SS/w - (LS/w)^2}$
2D	Độ lớn	$\ S_i, S_j\ $	$\sqrt{\mu_{S_i}^2 + \mu_{S_j}^2}$
2D	Bán kính	R_{S_i, S_j}	$\sqrt{(\sigma_{S_i}^2)^2 + (\sigma_{S_j}^2)^2}$
2D	Hiệp phương sai xấp xỉ	Cov_{S_i, S_j}	$SR_{ij}/(w_i + w_j)$
2D	Hệ số tương quan	P_{S_i, S_j}	$Cov_{S_i, S_j}/(\sigma_{S_i} \sigma_{S_j})$

Các thống kê chỉ cần một thống kê gia tăng được gọi là thống kê 1D; các thống kê cần hai thống kê gia tăng được gọi là thống kê 2D.

2. Các đặc trưng được trích xuất cho Kitsune

Mỗi khi một gói tin đến, hệ thống trích xuất một ảnh chụp hành vi của các máy và giao thức liên quan đến gói tin đó. Ảnh chụp gồm 115 thống kê lưu lượng phản ánh một cửa sổ thời gian ngắn về: (1) hành vi tổng quát của bên gửi gói tin; và (2) lưu lượng giữa bên gửi và bên nhận.

Cụ thể, các thống kê tóm tắt toàn bộ lưu lượng:

- phát sinh từ cùng địa chỉ MAC và IP nguồn của gói tin, ký hiệu **SrcMAC-IP**;
- phát sinh từ IP nguồn của gói tin, ký hiệu **SrcIP**;
- truyền giữa IP nguồn và IP đích của gói tin, ký hiệu **Channel**;
- truyền giữa socket TCP/UDP nguồn và đích của gói tin, ký hiệu **Socket**.

Từ một cửa sổ thời gian λ có thể trích xuất tổng cộng 23 đặc trưng. FE trích xuất cùng một tập đặc trưng trên năm cửa sổ: 100 ms, 500 ms, 1,5 giây, 10 giây và 1 phút trước thời điểm hiện tại, tương ứng $\lambda = 5, 3, 1, 0.1, 0.01$. Vì vậy tổng số đặc trưng là $23 \times 5 = 115$.

Không phải gói tin nào cũng áp dụng cho mọi loại kênh; chẳng hạn không có socket nếu gói không chứa datagram TCP hoặc UDP. Trong các trường hợp đó, các đặc trưng liên quan được đặt bằng 0. Do đó vector đặc trưng cuối $\vec{x}$ mà FE chuyển cho FM luôn thuộc R^n với $n = 115$.

Bảng II. Các đặc trưng được trích xuất trong mỗi cửa sổ λ khi một gói tin đến

Nhóm theo loại kênh

Loại kênh	Các cửa sổ λ	Thống kê trích xuất	Tổng số đặc trưng qua mọi cửa sổ	Ý nghĩa / ví dụ
MI	5, 3, 1, 0.1, 0.01	w_i, μ_i, σ_i	15	Toàn bộ lưu lượng phát sinh từ cùng MAC và IP, ví dụ (00:1C:B3:09:85:15, 192.168.0.2) $\rightarrow$ *
H	5, 3, 1, 0.1, 0.01	w_i, μ_i, σ_i	15	Toàn bộ lưu lượng phát sinh từ một IP, ví dụ 192.168.0.2 $\rightarrow$ *
HHjit	5, 3, 1, 0.1, 0.01	w_i, μ_i, σ_i	15	Độ chênh thời gian đến giữa hai IP, ví dụ 192.168.0.2 $\rightarrow$ 192.168.0.5
HH	5, 3, 1, 0.1, 0.01	$w_i, \mu_i, \sigma_i, \ S_i\ $	35	Lưu lượng dùng chung giữa hai địa chỉ IP, ví dụ 192.168.0.2 $\leftrightarrow$

Loại kênh	Các cửa sổ λ	Thống kê trích xuất	Tổng số đặc trưng qua mọi cửa sổ	Ý nghĩa / ví dụ
HpHp	5, 3, 1, 0.1, 0.01	$w_i, \mu_i, \sigma_i, \ S_i\ $	35	192.168.0.5 Lưu lượng dùng chung giữa hai socket, ví dụ 192.168.0.2:23523 ↔ 192.168.0.5:80

Nhóm theo thuộc tính gói tin

Thuộc tính	Thống kê	Nhóm gộp theo	Số đặc trưng trong một cửa sổ	Mô tả
Kích thước	μ_i, σ_i	SrcMAC-IP, SrcIP, Channel, Socket	8	Bảng thông lưu lượng đi ra
Kích thước	$\ S_i, S_j\ , R, C$	Channel, Socket	8	Bảng thông tổng hợp của lưu lượng đi ra và đi vào
Số lượng	w_i	SrcMAC-IP, SrcIP, Channel, Socket	4	Tốc độ gói tin đi ra
Jitter	w_i, μ_i, σ_i	Channel	3	Độ trễ giữa các gói tin đi ra

C. Bộ ánh xạ đặc trưng (FM)

Mục đích của FM là ánh xạ n đặc trưng - số chiều - của $\vec{x}$ thành k mẫu con nhỏ hơn, mỗi mẫu con tương ứng với một autoencoder trong tầng quần thể của AD. Ký hiệu tập có thứ tự của k mẫu con là:

$$v = \{\vec{v}_1, \vec{v}_2, \dots, \vec{v}_k\}.$$

Các mẫu con trong v có thể được xem là các không gian con của miền X của $\vec{x}$.

Để bảo đảm quần thể trong AD hoạt động hiệu quả với độ phức tạp thấp, phép ánh xạ được chọn $f(\vec{x}) = v$ phải:

1. Bảo đảm mỗi $\vec{v}_i$ có không quá m đặc trưng, trong đó m là tham số do người dùng định nghĩa. Tham số m ảnh hưởng đến độ phức tạp tổng thể của quần thể.
2. Ánh xạ mỗi trong số n đặc trưng của $\vec{x}$ đúng một lần vào các mẫu trong v , nhằm tránh làm quần thể quá rộng.
3. Tạo các không gian con của X có khả năng mô hình hóa hành vi bình thường đủ tốt để phát hiện sự kiện bất thường trong từng không gian con.
4. Được tìm ra bằng một quá trình trực tuyến, sao cho tại mỗi thời điểm không cần lưu quá một mẫu trong bộ nhớ.

Để đáp ứng các yêu cầu này, FM tìm phép ánh xạ f bằng cách phân cụm gia tăng các đặc trưng - các chiều - của X thành k nhóm, mỗi nhóm không lớn hơn m . Việc này được thực hiện bằng phân cụm phân cấp kết tụ trên dữ liệu tóm tắt được cập nhật gia tăng.

Cụ thể, thuật toán ánh xạ đặc trưng thực hiện:

1. Trong chế độ huấn luyện, cập nhật gia tăng các thống kê tóm tắt bằng đặc trưng của mẫu $\vec{x}$.
2. Khi chế độ huấn luyện kết thúc, thực hiện phân cụm phân cấp trên các thống kê để tạo f .
3. Trong chế độ thực thi, tính $f(\vec{x}_t) = v$ rồi chuyển v đến AD.

Để các đặc trưng được nhóm phản ánh hành vi bình thường, quá trình phân cụm dùng tương quan làm thước đo khoảng cách giữa hai chiều. Khoảng cách tương quan giữa hai vector u và v được định nghĩa:

$$d_{cor}(u, v) = 1 - \frac{(u - \hat{u}) \cdot (v - \hat{v})}{\|u - \hat{u}\|_2 \|v - \hat{v}\|_2},$$

trong đó $\hat{u}$ là trung bình các phần tử của u , còn $u \cdot v$ là tích vô hướng.

Sau đây là cách tóm tắt gia tăng khoảng cách tương quan giữa các đặc trưng để phục vụ phân cụm. Gọi n_t là số mẫu đã quan sát. Gọi $\vec{c}$ là vector n chiều chứa tổng tuyến tính của từng đặc trưng, trong đó:

$$c^{(i)} = \sum_{t=0}^{n_t} x_t^{(i)}.$$

Tương tự, gọi $\vec{c}_r$ là vector tổng phần dư của từng đặc trưng:

$$c_r^{(i)} = \sum_{t=0}^{n_t} \left(x_t^{(i)} - \frac{c^{(i)}}{n_t} \right),$$

và $\vec{c}_{rs}$ là vector tổng bình phương phần dư:

$$c_{rs}^{(i)} = \sum_{t=0}^{n_t} \left(x_t^{(i)} - \frac{c^{(i)}}{n_t} \right)^2.$$

Gọi C là ma trận tương quan từng phần $n \times n$:

$$[C_{i,j}] = \sum_{t=0}^{n_t} \left(x_t^{(i)} - \frac{c^{(i)}}{n_t} \right) \left(x_t^{(j)} - \frac{c^{(j)}}{n_t} \right),$$

đây là tổng tích giữa phần dư của đặc trưng i và j . Gọi D là ma trận khoảng cách tương quan giữa các đặc trưng của X . Từ C và $\vec{c}_{rs}$, có thể tính D tại mọi thời điểm:

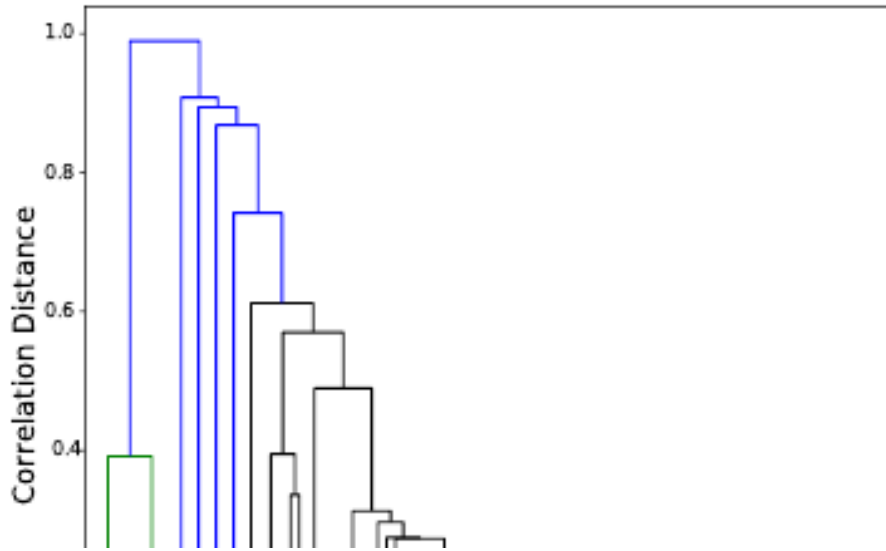
$$D = [D_{i,j}] = 1 - \frac{C_{i,j}}{\sqrt{c_{rs}^{(i)}} \sqrt{c_{rs}^{(j)}}}.$$

Khi đã có thể thu được D theo cách gia tăng, hệ thống thực hiện phân cụm phân cấp kết tụ trên D để tìm f . Thuật toán bắt đầu với n cụm, mỗi cụm chứa một điểm được biểu diễn bởi D . Tiếp theo, thuật toán tìm hai điểm gần nhất và hợp nhất các cụm tương ứng. Quá trình tìm và hợp nhất lặp lại cho đến khi chỉ còn một cụm lớn chứa toàn bộ n điểm. Cây biểu diễn các liên kết tìm được gọi là **dendrogram**, được minh họa trong Hình 4. Thông tin chi tiết về thuật toán phân cụm có thể xem trong [34]. Phân cụm phân cấp thường không phù hợp với bộ dữ liệu lớn do độ phức tạp cao; tuy nhiên, ma trận khoảng cách ở đây nhỏ, với n chỉ cỡ vài trăm, nên có thể tính trực tiếp tại thiết bị.

Từ dendrogram của D , có thể tìm k cụm đặc trưng sao cho không cụm nào lớn hơn m . Quy trình là cắt liên kết lớn nhất của dendrogram - nút trên cùng - rồi kiểm tra kích thước các cụm thu được. Nếu còn cụm lớn hơn m , tiếp tục cắt trong các cụm vượt giới hạn. Kết quả cuối là k nhóm đặc trưng có tương quan nội bộ mạnh và không nhóm nào vượt m . Các nhóm được lưu để thực hiện phép ánh xạ f .

Thuật toán này phù hợp cho xử lý trực tuyến và tại chỗ vì: (1) không bao giờ lưu quá một mẫu trong bộ nhớ; (2) dùng rất ít bộ nhớ, $O(n^2)$ trong chế độ huấn luyện; và (3) hoạt động nhanh vì mỗi lần cập nhật chỉ cần điều chỉnh một ma trận khoảng cách $n \times n$ nhỏ.

Hình 4. Ví dụ dendrogram của 115 đặc trưng được phân cụm từ một triệu gói tin mạng. Trục tung là khoảng cách tương quan, trục hoành là mã số chiều đặc trưng. Việc cắt cây theo giới hạn kích thước m tạo ra các nhóm đặc trưng đầu vào cho các autoencoder.



D. Bộ phát hiện bất thường (AD)

Như minh họa trong Hình 3, AD chứa một mạng nơ-ron đặc biệt được gọi là **KitNET** (Kitsune NETWORK). KitNET là một ANN không giám sát, được thiết kế cho phát hiện bất thường trực tuyến. KitNET gồm hai tầng autoencoder: **Tầng quần thể** và **Tầng đầu ra**.

Tầng quần thể (Ensemble Layer). Đây là một tập có thứ tự gồm k autoencoder ba tầng, mỗi autoencoder được ánh xạ với một mẫu con tương ứng trong v . Tầng này đo mức độ bất thường độc lập của từng không gian con - từng mẫu - trong v . Trong chế độ huấn luyện, các autoencoder học hành vi bình thường của không gian con tương ứng. Trong cả chế độ huấn luyện lẫn thực thi, mỗi autoencoder gửi sai số tái tạo RMSE của mình đến Tầng đầu ra.

Tầng đầu ra (Output Layer). Đây là một autoencoder ba tầng học các RMSE bình thường của Tầng quần thể trong chế độ huấn luyện. Tầng này tạo điểm bất thường cuối cùng, đồng thời xét: (1) quan hệ giữa các bất thường trong những không gian con; và (2) nhiễu tự nhiên trong lưu lượng mạng.

1. Khởi tạo

Khi AD nhận tập mẫu đã ánh xạ v đầu tiên từ FM, AD dùng v làm bản thiết kế để khởi tạo kiến trúc KitNET. Gọi θ là một autoencoder hoàn chỉnh; $L^{(1)}$ và $L^{(2)}$ lần lượt là Tầng quần thể và Tầng đầu ra. Tầng quần thể được định nghĩa:

$$L^{(1)} = \{\theta_1, \theta_2, \dots, \theta_k\}.$$

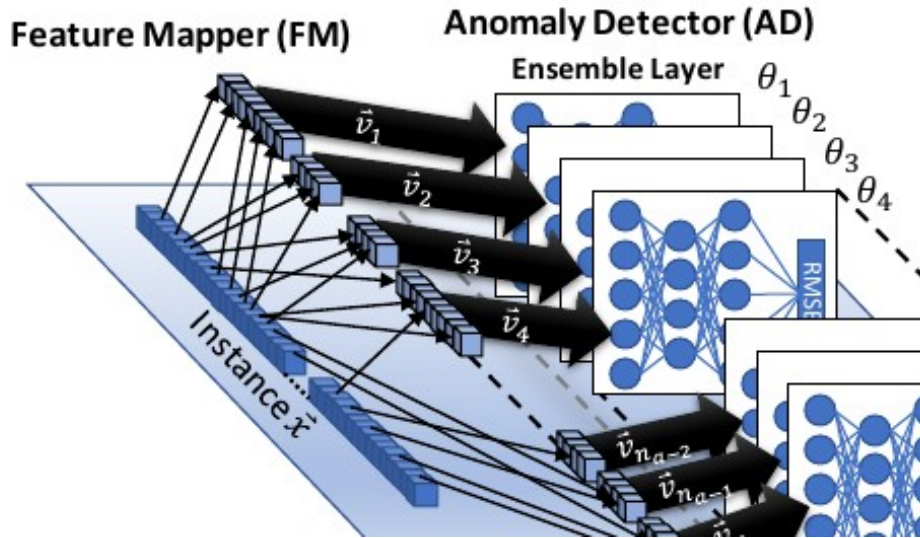
Mỗi autoencoder $\theta_i \in L^{(1)}$ có ba tầng nơ-ron: $\dim(\vec{v}_i)$ nơ-ron ở tầng đầu vào và tầng đầu ra, cùng $\lceil \beta \cdot \dim(\vec{v}_i) \rceil$ nơ-ron ở tầng giữa, trong đó $\beta \in (0, 1)$. Trong thực nghiệm, bài báo dùng $\beta = 3/4$. Hình 5 minh họa phép ánh xạ giữa $\vec{v}_i \in v$ và $\theta_i \in L^{(1)}$.

$L^{(2)}$ là một autoencoder duy nhất θ_0 , có k nơ-ron đầu vào và đầu ra, cùng $\lceil k \beta \rceil$ nơ-ron ở tầng giữa.

$L^{(1)}$ và $L^{(2)}$ không được nối bằng các liên kết có trọng số như ANN thông thường. Thay vào đó, đầu vào của $L^{(2)}$ là các tín hiệu sai số RMSE đã chuẩn hóa về 0-1 từ từng autoencoder tương ứng trong $L^{(1)}$. Việc truyền sai số tổng hợp RMSE của từng autoencoder, thay vì truyền tín hiệu từ từng nơ-ron riêng lẻ của $L^{(1)}$, làm giảm độ phức tạp của mạng. Trọng số của autoencoder θ_i được khởi tạo ngẫu nhiên theo phân phối đều:

$$U\left(-\frac{1}{\dim(\vec{v}_i)}, \frac{1}{\dim(\vec{v}_i)}\right).$$

Hình 5. Ánh xạ giữa FM và Tầng quần thể của KitNET. Một mẫu con $\vec{v}_i$ được ánh xạ từ $\vec{x}$ rồi chuyển đến autoencoder θ_i tương ứng.



2. Chế độ huấn luyện

Huấn luyện KitNET khác đôi chút so với huấn luyện ANN thông thường vì KitNET truyền tín hiệu sai số tái tạo RMSE giữa hai tầng chính. Ngoài ra, KitNET sử dụng SGD và chỉ học đúng một lần từ mỗi mẫu v quan sát được. Thuật toán huấn luyện một mẫu được trình bày dưới đây.

Thuật toán 4. Lan truyền ngược để huấn luyện KitNET

```

procedure train(L(1), L(2), v)
// Huấn luyện Tầng quần thể
1 z ← zeros(k)           // khởi tạo đầu vào cho L(2)
2 for  $\theta_i$  in L(1) do
3    $v_i' \leftarrow \text{norm0-1}(v_i)$ 
4    $A_i, y_i \leftarrow h\theta_i(v_i')$            // lan truyền tiến
5    $\text{deltasi} \leftarrow b\theta_i(v_i', y_i)$        // lan truyền ngược
6    $\theta_i \leftarrow \text{GD}\ell(A_i, \text{deltasi})$    // cập nhật trọng số
7    $z[i] \leftarrow \text{RMSE}(v_i', y_i)$          // đặt tín hiệu sai số
8 end
// Huấn luyện Tầng đầu ra
9  $z' \leftarrow \text{norm0-1}(z)$ 
10  $A_0, y_0 \leftarrow h\theta_0(z')$            // lan truyền tiến
11  $\text{deltas0} \leftarrow b\theta_0(z', y_0)$        // lan truyền ngược
12  $\theta_0 \leftarrow \text{GD}\ell(A_0, \text{deltas0})$    // cập nhật trọng số
13 return L(1), L(2)

```

Để chuẩn hóa 0-1 ở dòng 3, mỗi autoencoder phải lưu giá trị lớn nhất và nhỏ nhất từng quan sát đối với mỗi đặc trưng đầu vào. Các giá trị cực đại và cực tiểu này chỉ được cập nhật trong chế độ huấn luyện.

Tương tự phần III-E, KitNET phải được huấn luyện trên dữ liệu bình thường, không có tấn công. Đây là giả định phổ biến [35], [36] và có tính thực tế trong nhiều loại mạng, chẳng hạn hệ thống giám sát camera IP. Ngoài ra, có các phương pháp lọc dữ liệu huấn luyện để giảm tác động của những cuộc tấn công có thể đã tồn tại sẵn trong mạng [37], [38].

3. Chế độ thực thi

Trong chế độ thực thi, KitNET không cập nhật bất kỳ tham số nội bộ nào. Thay vào đó, nó thực hiện lan truyền tiến qua toàn bộ mạng và trả về sai số tái tạo RMSE của $L^{(2)}$.

Thuật toán 5. Thực thi KitNET

```

procedure execute(L(1), L(2), v)
// Thực thi Tầng quần thể
1 z ← zeros(k)           // khởi tạo đầu vào cho L(2)
2 for θi in L(1) do
3   vi' ← norm0-1(vi)
4   Ai, yi ← hθi(vi')      // lan truyền tiến
5   z[i] ← RMSE(vi', yi)    // đặt tín hiệu sai số
6 end
// Thực thi Tầng đầu ra
7 z' ← norm0-1(z)
8 A0, y0 ← hθ0(z')         // lan truyền tiến
9 return RMSE(z', y0)

```

Sai số tái tạo của $L^{(2)}$ đo mức độ bất thường của mẫu dựa trên quan hệ giữa các không gian con trong v . Ví dụ, xét hai autoencoder $\theta_i, \theta_j \in L^{(1)}$. Nếu RMSE của θ_i và θ_j có tương quan trong chế độ huấn luyện, thì việc mất tương quan trong chế độ thực thi có thể được coi là bất thường nghiêm trọng hơn so với chỉ cộng hai RMSE độc lập; điều ngược lại cũng tương tự. Do Tầng đầu ra $L^{(2)}$ học các quan hệ này và các quan hệ phức tạp khác trong chế độ huấn luyện, sai số tái tạo của nó đối với các RMSE từ $L^{(1)}$ sẽ phản ánh những bất thường đó.

4. Tính điểm bất thường

Đầu ra của KitNET là điểm bất thường RMSE $s \in \mathbb{R}$ như đã mô tả tại Phần III-E. s càng lớn thì mức độ bất thường càng cao. Để sử dụng s , cần xác định ngưỡng cắt ϕ . Cách đơn giản là đặt ϕ bằng điểm lớn nhất quan sát trong chế độ huấn luyện, với giả định mọi mẫu huấn luyện đều là lưu lượng bình thường.

Một cách khác là chọn ϕ theo xác suất. Cụ thể, có thể: (1) khớp các điểm RMSE đầu ra với phân phối log-normal hoặc một phân phối không chuẩn khác; và (2) phát cảnh báo khi xác suất xuất hiện của s rất thấp. Người dùng KitNET phải chọn cách xác định ϕ phù hợp với ứng dụng. Trong Phần V, năng lực phát hiện của Kitsune được đánh giá trực tiếp từ các điểm RMSE thô.

E. Độ phức tạp

Làm đường cơ sở so sánh, xét một autoencoder ba tầng duy nhất trên cùng không gian đặc trưng $\vec{x} \in \mathbb{R}^n$, với tỷ lệ tăng nén $\beta \in (0, 1)$. Như đã phân tích ở Phần III-F, độ phức tạp kích hoạt một tầng ANN là tích số nơ-ron của hai tầng kề nhau. Vì vậy, độ phức tạp thực thi autoencoder đơn là:

$$O(n \cdot \beta n + \beta n \cdot n) = O(n^2).$$

Với KitNET, k là số không gian con - số autoencoder - do FM lựa chọn, còn m là số đầu vào tối đa cho mỗi autoencoder trong $L^{(1)}$. Độ phức tạp thực thi $L^{(1)}$ và $L^{(2)}$ lần lượt là $O(k m^2)$ và $O(k^2)$. Vì $m \in \{1, 2, \dots, 10\}$ là tham số hằng của hệ thống, tổng độ phức tạp KitNET là:

$$O(k m^2 + k^2) = O(k^2).$$

Kết quả (13) cho thấy độ phức tạp của Tầng quần thể tăng tuyến tính theo n , nhưng độ phức tạp của Tầng đầu ra phụ thuộc số autoencoder - số không gian con - mà FM tạo ra. Trường hợp tốt nhất là $k=n/m$, khi đó hiệu năng tăng theo hệ số m . Trường hợp xấu nhất là FM gần như gán mỗi đặc trưng cho một autoencoder riêng trong $L^{(1)}$. Khi đó $k=n$, KitNET hoạt động như một autoencoder rộng duy nhất và không có lợi ích hiệu năng. Điều này cũng xảy ra nếu người dùng đặt $m=1$ hoặc $m=n$.

Tuy nhiên, trường hợp xấu này rất hiếm trên dữ liệu tự nhiên vì nó đòi hỏi dendrogram từ quá trình phân cụm của FM hoàn toàn mất cân bằng, chẳng hạn:

$$d_{cor}(x^{(1)}, x^{(2)}) < d_{cor}(x^{(2)}, x^{(3)}) < d_{cor}(x^{(3)}, x^{(4)}) < \dots$$

Trong đó x_i là chiều thứ i của R^n . Vì vậy, khi có nhiều đặc trưng, có thể kỳ vọng KitNET chạy nhanh hơn một autoencoder đơn hoặc stacked autoencoder. Độ phức tạp huấn luyện KitNET cũng là $O(k^2)$ vì hệ thống chỉ học một lần từ mỗi mẫu. Điều này khác với bộ phân loại dựa trên ANN, vốn thường phải chạy nhiều lượt - nhiều epoch - trên tập huấn luyện.

Hình 6. Hai camera trong mạng giám sát video IP. Bên trái: SNC-EM602RC. Bên phải: SNC-EB602R.



V. ĐÁNH GIÁ

Phần này đánh giá Kitsune về hiệu năng phát hiện và hiệu năng thời gian chạy. Trước hết, bài báo mô tả các bộ dữ liệu, tiếp theo là thiết lập thực nghiệm và cuối cùng trình bày kết quả.

A. Các bộ dữ liệu

Mục tiêu của Kitsune là cung cấp một IDS nhẹ, có thể xử lý nhiều gói tin mỗi giây trên bộ định tuyến đơn giản. Với mục tiêu đó, chúng tôi đánh giá khả năng phát hiện tấn công của Kitsune trên một mạng giám sát video bằng camera IP thực tế. Mạng này, được minh họa trong Hình 7, gồm hai khu vực triển khai, mỗi khu vực có bốn camera giám sát HD. Các camera được cấp nguồn qua PoE và kết nối với DVR bằng đường hầm VPN site-to-site. DVR tại địa điểm từ xa cho phép người dùng trên toàn cầu truy cập các luồng video thông qua kết nối VPN client-to-site. Các camera và cấu hình được mô tả trong Bảng IV; Hình 6 cho thấy hai trong số tám camera được sử dụng.

Có nhiều cuộc tấn công có thể được thực hiện đối với mạng giám sát video. Các cuộc tấn công nghiêm trọng nhất ảnh hưởng đến tính sẵn sàng và tính toàn vẹn của đường truyền video, chẳng hạn SYN flood vào camera mục tiêu hoặc tấn công người-ở-giữa chèn video vào luồng trực tiếp. Vì vậy, đánh giá tập trung vào các loại tấn công này. Bảng III tóm tắt các bộ dữ liệu tấn công; Hình 7 minh họa vị trí - vector - của kẻ tấn công cho từng tấn

công. Cột “Vi phạm” trong Bảng III cho biết thuộc tính bảo mật bị xâm phạm: tính bí mật (C), tính toàn vẹn (I) và tính sẵn sàng (A). Tất cả bộ dữ liệu được ghi tại điểm bắt gói tin chỉ ra trong Hình 7.

Để thiết lập nghe lén chủ động, chúng tôi dùng Raspberry Pi 3B làm cầu nối mạng vật lý. Pi được gắn bộ chuyển USB-to-Ethernet để có cổng Ethernet thứ hai, sau đó được đặt trực tiếp giữa tuyến cáp.

Đối với một số cuộc tấn công, gói tin độc hại không trực tiếp đi qua bộ định tuyến nơi Kitsune được kết nối. Trong các trường hợp này, FE vẫn phát hiện ngầm tấn công thông qua thay đổi thống kê trong hành vi mạng. Ví dụ, tấn công người-ở-giữa làm thay đổi thời gian truyền gói nhưng không nhất thiết thay đổi nội dung gói.

Để đánh giá Kitsune trên một mạng nhiều nhiều hơn, chúng tôi sử dụng thêm một mạng Wi-Fi có 9 thiết bị IoT và 3 máy tính. Các thiết bị IoT gồm bộ điều nhiệt, máy giám sát trẻ em, webcam, hai chuông cửa khác nhau và bốn camera an ninh giá rẻ. Trên mạng này, một camera an ninh được lây nhiễm bằng mẫu mã độc botnet Mirai thực tế.

Bảng III. Các bộ dữ liệu dùng để đánh giá Kitsune

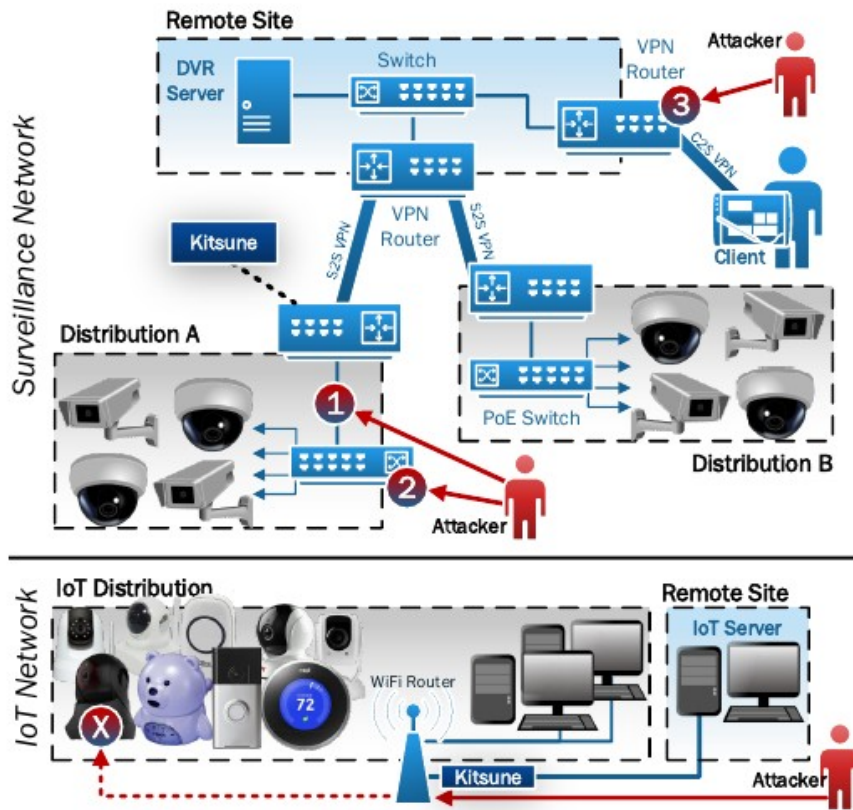
Loại tấn công	Tên tấn công	Công cụ	Mô tả hành động của kẻ tấn công	Vi phạm	Vector	Số gói tin	Huấn luyện (phút)	Thực thi (phút)
Trình sát	Quét hệ điều hành	Nmap	Quét mạng để tìm máy chủ và hệ điều hành, qua đó phát hiện lỗ hổng tiềm năng	C	1	1.697.851	33,3	18,9
Fuzzing	Fuzzing	SFuzz	Tìm lỗ hổng trong web server của camera bằng cách gửi lệnh ngẫu nhiên đến các CGI	C	3	2.244.139	33,3	52,2
Người-ở-giữa	Chèn video	Video Jack	Chèn một đoạn video đã ghi vào luồng video trực tiếp	C, I	1	2.472.401	14,2	19,2
Người-ở-giữa	ARP MitM	Ettercap	Chặn toàn bộ lưu lượng LAN bằng đầu độc ARP	C	1	2.504.267	8,05	20,1
Người-ở-giữa	Nghe lén chủ động	Raspberry Pi 3B	Chặn toàn bộ lưu lượng LAN bằng cầu nối mạng được bí mật gắn trên một đoạn cáp lộ thiên	C	2	4.554.925	20,8	74,8

Loại tấn công	Tên tấn công	Công cụ	Mô tả hành động của kẻ tấn công	Vi phạm	Vector	Số gói tin	Huấn luyện (phút)	Thực thi (phút)
Từ chối dịch vụ	SSDP Flood	Saddam	Làm quá tải DVR bằng cách khiến camera gửi dồn dập các thông báo UPnP đến máy chủ	A	1	4.077.266	14,4	26,4
Từ chối dịch vụ	SYN DoS	Hping3	Làm ngừng luồng video của camera bằng cách làm quá tải web server của camera	A	1	2.771.276	18,7	34,1
Từ chối dịch vụ	SSL Renegotiation	THC	Làm ngừng luồng video bằng cách gửi nhiều gói tái thương lượng SSL đến camera	A	1	6.084.492	10,7	54,9
Mã độc botnet	Mirai	Telnet	Lây nhiễm thiết bị IoT bằng Mirai qua thông tin đăng nhập mặc định, rồi quét tìm nạn nhân để bị tấn công mới	C, I	X	764.137	52,0	66,9

Bảng IV. Thông số và thống kê của các camera dùng trong thực nghiệm

Mẫu camera	Độ phân giải	Codec	FPS	Gói/giây trung bình	Bảng thông trung bình	Giao thức
SNC-EM602RC	1280×720	H.264/MPEG4	15	195	1,8 Mbit/s	RTP
SNC-EB600	1280×720	H.264/MPEG4	15	290	1,8 Mbit/s	HTTPS/TCP (TLSv1)
SNC-EM600	1280×720	H.264/MPEG4	15	350	1,4 Mbit/s	RTP
SNC-EB602R	1280×720	H.264/MPEG4	15	320	1,8 Mbit/s	HTTP/TCP

Hình 7. Cấu trúc mạng trong thực nghiệm. Phần trên là mạng giám sát, gồm hai khu vực camera, switch PoE, router VPN, DVR, máy khách, Kitsune và ba vị trí kẻ tấn công được ký hiệu 1-3. Phần dưới là mạng IoT, gồm các thiết bị IoT, router Wi-Fi, máy chủ IoT, Kitsune và vị trí tấn công X.



B. Thiết lập thực nghiệm

Thuật toán ngoại tuyến thường hoạt động tốt hơn thuật toán trực tuyến vì trong khi huấn luyện chúng có quyền truy cập toàn bộ dữ liệu và có thể lặp nhiều lần trên dữ liệu. Tuy nhiên, thuật toán trực tuyến hữu ích khi tài nguyên như sức mạnh tính toán và bộ nhớ bị giới hạn. Trong đánh giá, chúng tôi so sánh Kitsune với cả thuật toán trực tuyến và ngoại tuyến. Các thuật toán trực tuyến tạo đường cơ sở để so sánh Kitsune như một bộ phát hiện bất thường trực tuyến; các thuật toán ngoại tuyến cho góc nhìn gần như giới hạn trên của hiệu năng Kitsune.

Làm đường cơ sở bổ sung, chúng tôi đánh giá **Suricata** [39], một NIDS dựa trên chữ ký. Suricata là NIDS mã nguồn mở tương tự Snort nhưng được song song hóa trên nhiều luồng. NIDS dựa trên chữ ký có tỷ lệ dương tính giả thấp hơn đáng kể so với NIDS dựa trên bất thường, nhưng không thể phát hiện đe dọa chưa biết hoặc hành vi bất thường/trừu tượng. Suricata được cấu hình với 13.465 luật từ kho Emerging Threats [40].

Đối với thuật toán ngoại tuyến, chúng tôi dùng **Isolation Forest (IF)** [41] và **Gaussian Mixture Model (GMM)** [42]. IF là phương pháp phát hiện ngoại lai dựa trên quần thể, còn GMM là phương pháp thống kê dựa trên thuật toán expectation-maximization. Đối với thuật toán trực tuyến, chúng tôi dùng GMM gia tăng từ [43] và **pcStream2** [44]. pcStream2 là thuật toán phân cụm dòng dữ liệu, phát hiện ngoại lai bằng khoảng cách Mahalanobis giữa mẫu mới và các cụm đã biết.

Trong mỗi thực nghiệm - mỗi bộ dữ liệu - tất cả thuật toán được huấn luyện trên một triệu gói tin đầu tiên rồi thực thi trên phần còn lại. Thời lượng tương ứng của một triệu gói đầu phụ thuộc tốc độ mạng tại thời điểm bắt gói. Ví dụ, với bộ OS Scan, thuật toán được huấn luyện trên một triệu gói đầu rồi thực thi trên 697.851 gói còn lại. Bảng III liệt kê thời gian huấn luyện và thực thi tương đối của từng bộ dữ liệu. Tất cả thuật toán nhận cùng một tập đặc trưng trong Bảng II.

Kitsune có một tham số chính $m \in \{1, 2, \dots, n\}$, là số đầu vào tối đa của mỗi autoencoder trong quần thể KitNET. Trong đánh giá hiệu năng phát hiện, chúng tôi dùng $m=1$ và $m=10$. Các thuật toán khác dùng thiết lập mặc định.

C. Chỉ số đánh giá

Đầu ra của bộ phát hiện bất thường là điểm $s \in \mathbb{R}$; giá trị càng lớn biểu thị bất thường càng nghiêm trọng, ví dụ RMSE của autoencoder. Điểm này thường được chuẩn hóa sao cho giá trị dưới 1 là bình thường và trên 1 là bất thường. Việc chuẩn hóa được thực hiện bằng cách chia s cho ngưỡng cắt ϕ . Cách chọn ϕ ảnh hưởng rất lớn đến hiệu năng thuật toán.

Hiệu năng phát hiện trên một bộ dữ liệu có thể đo bằng dương tính thật (TP), âm tính thật (TN), dương tính giả (FP) và âm tính giả (FN). Trong đánh giá, bài báo đo:

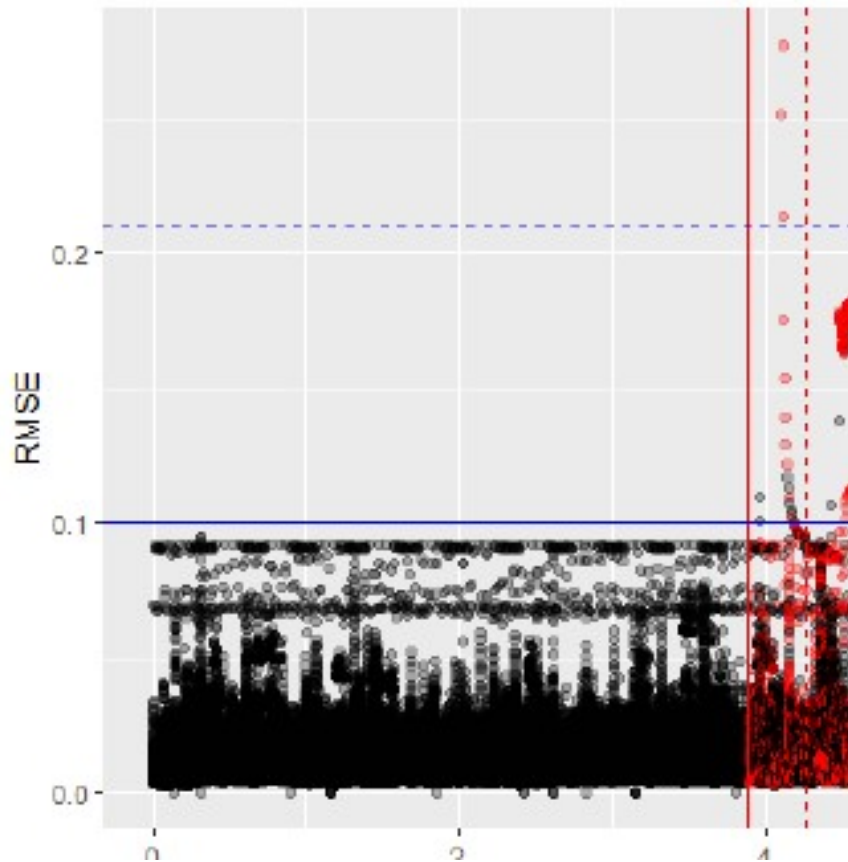
$$TPR = \frac{TP}{TP + FN}, FNR = \frac{FN}{FN + TP}, FPR = \frac{FP}{FP + TN}.$$

TPR và FNR được tính khi chọn ϕ sao cho FPR rất thấp, cụ thể bằng 0,001. Bài báo cũng đếm số dương tính thật khi FPR bằng 0. Các chỉ số này phản ánh số gói độc hại được phát hiện đúng và số gói bị bỏ sót khi có rất ít hoặc không có cảnh báo giả. Trong phát hiện xâm nhập mạng, số cảnh báo giả phải được giảm tối đa vì chuyên gia phải tốn thời gian và chi phí để điều tra từng cảnh báo.

Hình 8 biểu diễn điểm bất thường KitNET trước và trong tấn công fuzzing. Đường xanh phía dưới là ngưỡng thấp nhất có thể chọn trong pha huấn luyện mà không tạo FP. Đường xanh phía trên là ngưỡng thấp nhất trong thời gian tấn công mà vẫn không tạo FP - gần giống một tối ưu toàn cục. Hai ngưỡng có thể được xem như trường hợp xấu nhất và tốt nhất của việc chọn ngưỡng. Đo hiệu năng tại cả hai ngưỡng giúp hiểu rõ hơn tiềm năng của thuật toán phát hiện bất thường.

Để đo hiệu năng tổng quát trên mọi giá trị ϕ , bài báo dùng diện tích dưới đường ROC (AUC) và tỷ lệ lỗi bằng nhau (EER). Trong bối cảnh này, AUC là xác suất bộ phân loại xếp một mẫu bất thường được chọn ngẫu nhiên cao hơn một mẫu bình thường được chọn ngẫu nhiên. AUC bằng 1 biểu thị bộ phát hiện hoàn hảo; AUC bằng 0,5 tương đương đoán ngẫu nhiên. EER phản ánh sự đánh đổi giữa FNR và FPR, được tính tại điểm FNR và FPR nhỏ nhất và bằng nhau.

Hình 8. Điểm bất thường RMSE của KitNET trước và trong một cuộc tấn công fuzzing. Hai đường đỏ chỉ thời điểm kẻ tấn công kết nối vào mạng và thời điểm bắt đầu tấn công. Các đường ngưỡng màu xanh minh họa ngưỡng không tạo đường tính giả trong pha huấn luyện và ngưỡng tối ưu trong toàn bộ khoảng quan sát.



D. Hiệu năng phát hiện

Hình 9 trình bày TPR và FNR của từng thuật toán trên từng bộ dữ liệu khi ngưỡng được chọn để FPR bằng 0 và 0,001; đồng thời trình bày AUC và EER. GMM và Isolation Forest là thuật toán theo lô - ngoại tuyến - có quyền truy cập toàn bộ dữ liệu và có thể lặp nhiều lần. Vì vậy, chúng đóng vai trò mục tiêu hiệu năng gần tối ưu mà thuật toán trực tuyến hướng tới.

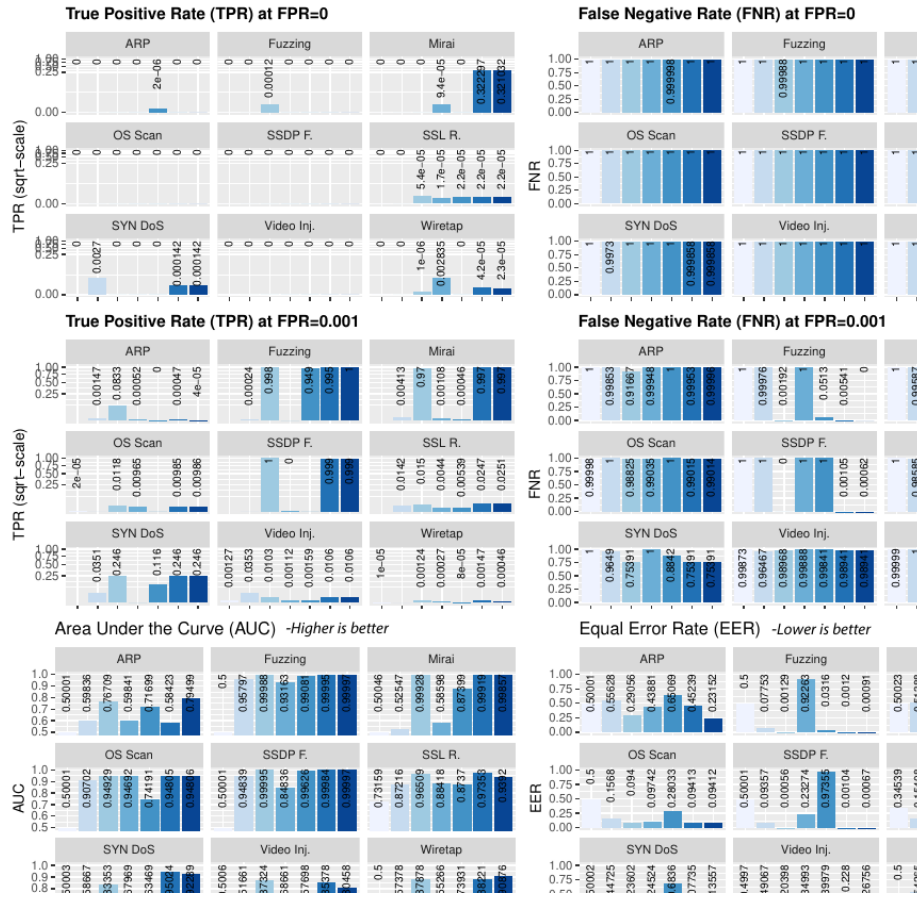
Kết quả cho thấy Kitsune hoạt động rất tốt so với các thuật toán ngoại tuyến. Cụ thể, Kitsune thậm chí tốt hơn GMM trong việc phát hiện nghe lén chủ động. Thuật toán cũng đạt EER tốt hơn GMM trên các bộ ARP, Fuzzing, Mirai, SSL Renegotiation, SYN và Active Wiretap.

Hình 9 cũng cho thấy sự đánh đổi giữa hiệu năng phát hiện và m , tức hiệu năng thời gian chạy. Người dùng ưu tiên phát hiện hơn tốc độ nên chọn m gần 1 hoặc gần n ; người dùng ưu tiên tốc độ nên chọn m ở mức trung bình. Kitsune cho phép điều chỉnh tham số theo yêu cầu hệ thống. Ảnh hưởng của m đến thời gian chạy được trình bày tại Phần V-E.

So với các thuật toán trực tuyến GMM gia tăng và pcStream2, Kitsune nhìn chung vượt trội về cả AUC và EER.

Hàng trên của Hình 9 biểu diễn số dương tính thật tối đa mà mỗi thuật toán đạt được khi ngưỡng được đặt sao cho không có dương tính giả. Nói cách khác, các biểu đồ cho thấy khả năng đẩy điểm bất thường của gói độc hại lên trên “nền nhiễu”. Kitsune phát hiện các cuộc tấn công tốt hơn các thuật toán khác trên toàn bộ bộ dữ liệu, và trong phần lớn trường hợp tốt hơn cả GMM. Đôi khi Kitsune với $m=10$ tốt hơn $m=1$ vì autoencoder Tăng đầu ra làm giảm nền nhiễu của quần thể; hiệu ứng này có thể khuếch đại điểm của một số ngoại lai.

Hình 9. Kết quả thực nghiệm của tất cả thuật toán trên từng bộ dữ liệu. Bốn nhóm chỉ số gồm: TPR khi $FPR = 0,001$; FNR khi $FPR = 0,001$; AUC; và EER. Chú giải gồm Suricata (dựa trên chữ ký), Isolation Forest và GMM (bất thường ngoại tuyến), GMM gia tăng và pcStream (bất thường trực tuyến), Kitsune với $m = 10$ và $m = 1$. TPR/AUC càng cao càng tốt; FNR/EER càng thấp càng tốt.



E. Hiệu năng thời gian chạy

Một trong những ưu điểm lớn nhất của Kitsune là hiệu năng thời gian chạy. Như phân tích tại Phần IV-E, quần thể autoencoder nhỏ của KitNET hiệu quả hơn một autoencoder duy nhất vì làm giảm tổng số phép toán cần thiết để xử lý mỗi mẫu.

Để chứng minh, chúng tôi đo chuẩn trên Raspberry Pi 3B và máy ảo Ubuntu Linux chạy trên máy tính Windows 10; thông số đầy đủ trong Bảng V. Thực nghiệm được viết bằng C++, dùng $n = 198$ đặc trưng thống kê của gói tin và chạy trên một lõi - lõi vật lý trên Pi và lõi logic trên máy ảo Ubuntu.

Hình 10 biểu diễn ảnh hưởng của kích thước Tăng quần thể đến tốc độ xử lý gói tin. Với một autoencoder duy nhất trong $L^{(1)}$, Pi và PC lần lượt xử lý khoảng 1.000 và 7.500 gói/giây. Khi có 35 autoencoder trong $L^{(1)}$, hiệu năng của cả hai môi trường tăng khoảng năm lần, lần lượt đạt khoảng 5.400 và 37.300 gói/giây.

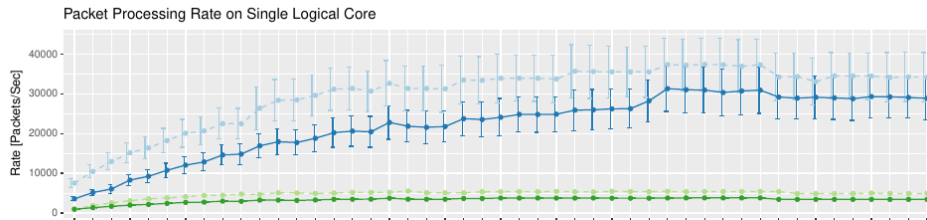
Hình 11 quan sát chi tiết thời gian xử lý trên Pi với $k = 1$ và $k = 35$. Quần thể không chỉ tăng tốc mà còn làm giảm phương sai thời gian xử lý, có thể hữu ích trong những ứng dụng không mong muốn jitter của lưu lượng mạng.

Kết quả đo chuẩn trên Raspberry Pi cho thấy một bộ định tuyến đơn giản, tài nguyên hạn chế, vẫn có thể chạy Kitsune như một NIDS. Vì vậy, Kitsune là giải pháp NIDS phân tán có chi phí thấp và đáng tin cậy. Do thực nghiệm chỉ chạy trên một lõi, tốc độ xử lý còn có thể tăng thêm. Công việc tương lai là song song hóa KitNET trên nhiều lõi.

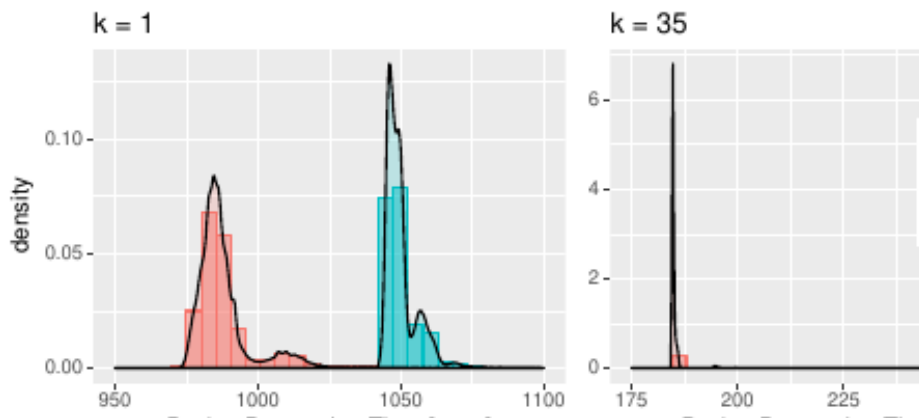
Bảng V. Môi trường dùng để đo chuẩn

Thuộc tính	Môi trường 1: Raspberry Pi 3B	Môi trường 2: Máy ảo Ubuntu
CPU	Broadcom BCM2837	Intel i7-4790
Xung nhịp	1,2 GHz	3,60 GHz
Số lõi	4	4 lõi, 8 luồng logic
RAM	1 GB	4 GB

Hình 10. Ảnh hưởng của kích thước Tầng queue thể k đến tốc độ xử lý gói trung bình. Đo trên một lõi của Raspberry Pi và máy ảo Ubuntu với $n=198$ đặc trưng; biểu đồ phân biệt chế độ huấn luyện và thực thi. Tốc độ tăng mạnh khi k tăng từ 1 đến khoảng 35.



Hình 11. Phân bố thời gian xử lý gói trên Raspberry Pi. Biểu đồ trên ứng với $k=1$, biểu đồ dưới ứng với $k=35$; cả chế độ huấn luyện và thực thi đều được thể hiện. Khi $k=35$, thời gian xử lý thấp hơn và phân bố hẹp hơn đáng kể.



VI. TẤN CÔNG ĐỐI KHÁNG VÀ BIỆN PHÁP ỨNG PHÓ

Khi sử dụng Kitsune, cần cân nhắc một số khía cạnh. Trước hết, một đối thủ nâng cao có thể thực hiện học máy đối kháng [45]. Khi mới được cài đặt, Kitsune giả định toàn bộ lưu lượng trong chế độ huấn luyện là lành tính. Vì vậy, một đối thủ đã tồn tại sẵn trong mạng có thể né tránh sự phát hiện của Kitsune. Tuy nhiên, trong chế độ thực thi, Kitsune sẽ phát hiện các cuộc tấn công mới và các mối đe dọa mới khi chúng xuất hiện. Dù vậy, người dùng phải nhận thức rủi ro này khi cài Kitsune trên mạng có khả năng đã bị xâm nhập.

Một hướng nghiên cứu tương lai là xây dựng cơ chế lọc an toàn các mẫu có thể bị nhiễm bẩn trong quá trình huấn luyện. Ví dụ, hệ thống có thể thực thi mô hình trước và kiểm tra điểm bất thường. Nếu điểm cao, mẫu đó không được dùng để huấn luyện vì mục tiêu là chỉ học từ mẫu lành tính. Theo cách này, hệ thống có thể duy trì chế độ huấn luyện vô thời hạn.

Nếu có lo ngại lớn rằng mạng mục tiêu đã bị xâm nhập, người dùng có thể ưu tiên NIDS dựa trên chữ ký như Snort. Sự đánh đổi là NIDS chữ ký không thể tự động phát hiện mối đe dọa mới hoặc hành vi trừu tượng, như kết quả đánh giá đã chứng minh. Một phương án thỏa hiệp tốt là cài một NIDS hiệu quả như Snort 3.0 hoặc Suricata song song với Kitsune.

Một mối đe dọa khác là tấn công DoS nhằm vào FE. Trong kịch bản này, kẻ tấn công gửi nhiều gói tin với địa chỉ IP ngẫu nhiên. FE sẽ tạo rất nhiều thống kê gia tăng, cuối cùng tiêu thụ hết bộ nhớ thiết bị. Mặc dù cuộc tấn công tạo điểm bất thường lớn, hệ thống có thể trở nên mất ổn định. Vì vậy, người dùng được khuyến nghị giới hạn số thống kê gia tăng được phép lưu trong bộ nhớ.

Với bản cài đặt Kitsune bằng C++, khoảng 1 MB RAM có thể chứa xấp xỉ 1.000 liên kết mạng, giả định mỗi liên kết có năm cửa sổ suy giảm. Một giải pháp để duy trì mức sử dụng bộ nhớ nhỏ là định kỳ tìm và xóa các thống kê gia tăng có $w_i \approx 0$. Trong thực tế, phần lớn thống kê gia tăng ở trạng thái này vì hệ thống dùng các giá trị λ tương đối lớn, khiến dữ liệu suy giảm nhanh.

VII. KẾT LUẬN

Kitsune là một NIDS dựa trên mạng nơ-ron, được thiết kế để hoạt động hiệu quả và cắm-là-chạy. Hệ thống đạt mục tiêu này bằng cách theo dõi hiệu quả hành vi của toàn bộ các kênh mạng và sử dụng một quần thể autoencoder - KitNET - để phát hiện bất thường. Bài báo đã trình bày chi tiết quá trình học máy trực tuyến của khung và đánh giá cả hiệu năng phát hiện lẫn hiệu năng thời gian chạy.

KitNET, dù là thuật toán trực tuyến, hoạt động gần tương đương các thuật toán theo lô hoặc ngoại tuyến, và trong một số trường hợp còn tốt hơn. Hơn nữa, thuật toán đủ hiệu quả để chạy trên một lõi của Raspberry Pi và có tiềm năng cao hơn nữa trên CPU mạnh hơn.

Tóm lại, khả năng triển khai một NIDS thông minh trên thiết bị mạng đơn giản mang lại lợi ích lớn, đặc biệt khi toàn bộ quá trình triển khai là cắm-là-chạy. Các tác giả kỳ vọng Kitsune hữu ích cho cả chuyên gia thực hành và nhà nghiên cứu, đồng thời thuật toán KitNET sẽ thúc đẩy sự quan tâm đến việc phát triển lĩnh vực phát hiện bất thường trực tuyến dựa trên mạng nơ-ron.

LỜI CẢM ƠN

Các tác giả cảm ơn Masayuki Nakae thuộc NEC Corporation of America vì những phản hồi và sự hỗ trợ trong việc xây dựng hệ thống camera giám sát. Các tác giả cũng cảm ơn Yael Mathov, Michael Bohadana và Yishai Wiesner vì đã hỗ trợ tạo bộ dữ liệu Mirai.

TÀI LIỆU THAM KHẢO

- [1] Marshall A Kuypers, Thomas Maillart, and Elisabeth Paté-Cornell. An empirical analysis of cyber security incidents at a large organization. Department of Management Science and Engineering, Stanford University, School of Information, UC Berkeley, 30, 2016.
- [2] Dimitrios Damopoulos, Sofia A Menesidou, Georgios Kambourakis, Maria Papadaki, Nathan Clarke, and Stefanos Gritzalis. Evaluation of anomaly-based ids for mobile devices using machine learning classifiers. Security and Communication Networks, 5(1):3–14, 2012.
- [3] Yinhui Li, Jingbo Xia, Silan Zhang, Jiakai Yan, Xiaochuan Ai, and Kuobin Dai. An efficient intrusion detection system based on support vector machines and gradually feature removal method. Expert Systems with Applications, 39(1):424–430, 2012. 14
- [4] Supranamaya Ranjan. Machine learning based botnet detection using real-time extracted traffic features, March 25 2014. US Patent 8,682,812.
- [5] Varun Chandola, Arindam Banerjee, and Vipin Kumar. Anomaly detection: A survey. ACM computing surveys (CSUR), 41(3):15, 2009.
- [6] Howard B Demuth, Mark H Beale, Orlando De Jess, and Martin T Hagan. Neural network design. Martin Hagan, 2014.
- [7] Nidhi Srivastav and Rama Krishna Challa. Novel intrusion detection system integrating layered framework with neural network. In Advance Computing Conference (IACC), 2013 IEEE 3rd International, pages 682–689. IEEE, 2013.

- [8] Reyadh Shaker Naoum, Namh Abdula Abid, and Zainab Namh Al- Sultani. An enhanced resilient backpropagation artificial neural network for intrusion detection system. *International Journal of Computer Science and Network Security (IJCSNS)*, 12(3):11, 2012.
- [9] Chunlin Zhang, Ju Jiang, and Mohamed Kamel. Intrusion detection using hierarchical neural networks. *Pattern Recognition Letters*, 26(6):779–791, 2005.
- [10] Rebecca Petersen. Data mining for network intrusion detection: A comparison of data mining algorithms and an analysis of relevant features for detecting cyber-attacks, 2015.
- [11] Rupesh K Srivastava, Klaus Greff, and Jürgen Schmidhuber. Training very deep networks. In *Advances in neural information processing systems*, pages 2377–2385, 2015.
- [12] Jonathan A Silva, Elaine R Faria, Rodrigo C Barros, Eduardo R Hruschka, André CPLF de Carvalho, and João Gama. Data stream clustering: A survey. *ACM Computing Surveys (CSUR)*, 46(1):13, 2013.
- [13] Garcia-Teodoro et. al. Anomaly-based network intrusion detection: Techniques, systems and challenges. *computers & security*, 28(1):18– 28, 2009.
- [14] Harjinder Kaur, Gurpreet Singh, and Jaspreet Minhas. A review of machine learning based anomaly detection techniques. *arXiv preprint arXiv:1307.7286*, 2013.
- [15] Taeshik Shon and Jongsub Moon. A hybrid machine learning approach to network anomaly detection. *Information Sciences*, 177(18):3799– 3821, 2007.
- [16] Taeshik Shon, Yongdae Kim, Cheolwon Lee, and Jongsub Moon. A machine learning framework for network anomaly detection using svm and ga. In *Information Assurance Workshop, 2005. IAW’05. Proceedings from the Sixth Annual IEEE SMC*, pages 176–183. IEEE, 2005.
- [17] Anna L Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016.
- [18] Ke Wang and Salvatore J Stolfo. Anomalous payload-based network intrusion detection. In *RAID*, volume 4, pages 203–222. Springer, 2004.
- [19] Miao Xie, Jiankun Hu, Song Han, and Hsiao-Hwa Chen. Scalable hypergrid k-nn-based online anomaly detection in wireless sensor networks. *IEEE Transactions on Parallel and Distributed Systems*, 24(8):1661–1670, 2013.
- [20] Srinivas Mukkamala, Andrew H Sung, and Ajith Abraham. Intrusion detection using an ensemble of intelligent paradigms. *Journal of network and computer applications*, 28(2):167–182, 2005.
- [21] Xiangzeng Zhou, Lei Xie, Peng Zhang, and Yanning Zhang. An ensemble of deep neural networks for object tracking. In *Image Processing (ICIP), 2014 IEEE International Conference on*, pages 843– 847. IEEE, 2014.
- [22] Mahmood Yousefi-Azar, Vijay Varadharajan, Len Hamey, and Uday Tupakula. Autoencoder-based feature learning for cyber security applications. In *Neural Networks (IJCNN), 2017 International Joint Conference on*, pages 3854–3861. IEEE, 2017.
- [23] Ahmad Javaid, Quamar Niyaz, Weiqing Sun, and Mansoor Alam. A deep learning approach for network intrusion detection system. In *Proceedings of the 9th EAI International Conference on Bio-inspired Information and Communications Technologies (formerly BIONETICS)*, pages 21–26. ICST (Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering), 2016.
- [24] Mayu Sakurada and Takehisa Yairi. Anomaly detection using autoencoders with nonlinear dimensionality reduction. In *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, page 4. ACM, 2014.
- [25] T Jayalakshmi and A Santhakumaran. Statistical normalization and back propagationfor classification. *International Journal of Computer Theory and Engineering*, 3(1):89, 2011.
- [26] Suvrit Sra, Sebastian Nowozin, and Stephen J Wright. *Optimization for machine learning*. Mit Press, 2012.
- [27] Léon Bottou. Stochastic gradient descent tricks. In *Neural networks: Tricks of the trade*, pages 421–436. Springer, 2012.
- [28] Guang-Bin Huang, Qin-Yu Zhu, and Chee-Kheong Siew. Extreme learning machine: a new learning scheme of feedforward neural networks. In *Neural Networks, 2004. Proceedings. 2004 IEEE International Joint Conference on*, volume 2, pages 985–990. IEEE, 2004.
- [29] Jacob Walker, Carl Doersch, Abhinav Gupta, and Martial Hebert. An uncertain future: Forecasting from static images using variational autoencoders. In *European Conference on Computer Vision*, pages 835– 851. Springer, 2016.
- [30] Pascal Vincent, Hugo Larochelle, Isabelle Lajoie, Yoshua Bengio, and Pierre-Antoine Manzagol. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion. *Journal of Machine Learning Research*, 11(Dec):3371–3408, 2010.
- [31] Yusuke Sugiyama and Kunio Goto. Design and implementation of a network emulator using virtual network stack. In *7th International Symposium on Operations Research and Its Applications (ISORA08)*, pages 351–358, 2008.
- [32] Eric Leblond and Giuseppe Longo. *Suricata idps and its interaction with linux kernel*.

- [33] Borja Merino. Instant Traffic Analysis with Tshark How-to. Packt Publishing Ltd, 2013.
- [34] Fionn Murtagh and Pedro Contreras. Methods of hierarchical clustering. arXiv preprint arXiv:1105.0121, 2011.
- [35] Wenke Lee, Salvatore J Stolfo, et al. Data mining approaches for intrusion detection. In USENIX Security Symposium, pages 79–93. San Antonio, TX, 1998.
- [36] Mingrui Wu and Jieping Ye. A small sphere and large margin approach for novelty detection using training data with outliers. IEEE transactions on pattern analysis and machine intelligence, 31(11):2088– 2092, 2009.
- [37] Niladri Sett, Subhrendu Chattopadhyay, Sanasam Ranbir Singh, and Sukumar Nandi. A time aware method for predicting dull nodes and links in evolving networks for data cleaning. In Web Intelligence (WI), 2016 IEEE/WIC/ACM International Conference on, pages 304–310. IEEE, 2016.
- [38] Deepthy K Denatious and Anita John. Survey on data mining techniques to enhance intrusion detection. In Computer Communication and Informatics (ICCCI), 2012 International Conference on, pages 1–5. IEEE, 2012.
- [39] Suricata — open source ids / ips / nsm engine. <https://suricata-ids.org/>, 11 2017. (Accessed on 11/14/2017).
- [40] Index of /open/suricata/rules. <https://rules.emergingthreats.net/open/suricata/rules/>, 11 2017. (Accessed on 11/14/2017).
- [41] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In Data Mining, 2008. ICDM’08. Eighth IEEE International Conference on, pages 413–422. IEEE, 2008.
- [42] Douglas Reynolds. Gaussian mixture models. Encyclopedia of biometrics, pages 827–832, 2015.
- [43] Sylvain Calinon and Aude Billard. Incremental learning of gestures by imitation in a humanoid robot. In Proceedings of the ACM/IEEE international conference on Human-robot interaction, pages 255–262. ACM, 2007.
- [44] Yisroel Mirsky, Tal Halpern, Rishabh Upadhyay, Sivan Toledo, and Yuval Elovici. Enhanced situation space mining for data streams. In Proceedings of the Symposium on Applied Computing, pages 842–849. ACM, 2017.
- [45] Ling Huang, Anthony D Joseph, Blaine Nelson, Benjamin IP Rubinstein, and JD Tygar. Adversarial machine learning. In Proceedings of the 4th ACM workshop on Security and artificial intelligence, pages 43–58. ACM, 2011.